

DIGITAL WALLET AND PAYMENT GATEWAY SIMULATOR

Secure Software Engineering End Semester Capstone Laboratory Report

Course Code: 24CYS401 — Secure Software Engineering

Candidate Name / Student ID	Sathvik Valivety (GitHub: sathvikvalivety)
Degree & Program	B.Tech Computer Science and Engineering (Cyber Security)
Institution / Department	Amrita Vishwa Vidyapeetham, Department of Cyber Security
Examination / Semester	24CYS401 End Semester Laboratory Examination
Academic Year / Date	October 2026 Academic Session 2026–2027
Repository URL	https://github.com/sathvikvalivety/digital-wallet-and-payment-gateway-simulator

Executive Summary

The Digital Wallet and Payment Gateway Simulator (DWPG) is an end-to-end, high-integrity financial transaction simulator engineered in compliance with rigorous secure software engineering standards for the 24CYS401 End Semester Laboratory Examination. The primary objective is to simulate a complete electronic payment ecosystem —spanning user onboarding, credential issuance, digital wallet provisioning, atomic top-ups, merchant registration, idempotent checkout settlement, stateful refunds, and immutable double-entry ledger bookkeeping —while systematically resolving critical distributed financial security challenges: race condition double-spending, replay attacks, parameter tampering, broken authorization, and forensic audit evasion.



Curricular Examination Compliance

This report documents complete empirical evidence for all 16 prescribed phases of the Secure Software Engineering examination. All 24 architectural, UML, data flow, and threat diagrams were authored as editable draw.io sources (docs/diagrams/drawio/) and exported via the official draw.io CLI. 15 real Playwright UI screenshots, 16 authentic Jira figures, 7 SonarQube SAST screenshots, and verified Kubernetes runtime workloads are embedded as immutable proof.

Core System Metrics at a Glance

Engineering Dimension	Implemented Specification	Empirical Verification Status
Target Platform	Java 21 LTS, Spring Boot 3.3.4, React 18, MariaDB 11.4	Operational on localhost:8080 & localhost:3000
Containerization & K8s	Multi-stage Docker, Minikube Namespace 'dwpg'	Non-root UID 10001, NetworkPolicy isolate-mariadb active
Scrum Cadence	2 Sprints (52 Story Points committed, 47 delivered)	Jira Board #101, 13 user stories, defect DEF-001 resolved
Automated Test Suite	42 Tests: 25 JUnit 5 + 9 REST Integration + 8 Security	100% Passed (0 Failures, 0 Errors) in 4.281s
SAST Quality Gate	SonarQube 9.9.8 LTS (dwpg-simulator)	Quality Gate: OK 0 Vulnerabilities, 0 Hotspots, 0 Bugs, 62.6% Coverage
Draw.io Source Diagrams	24 Diagrams authored in Draw.io / Diagrams.net	All 24 available in docs/diagrams/drawio/, PNG & SVG exported

1. Project Overview and Context

1.1 Case Study Context

Modern digital payment gateways and electronic wallets form the lifeblood of contemporary e-commerce. However, implementing financial platforms introduces severe systemic risks: network retries leading to duplicate debits, race conditions allowing users to double-spend simultaneous balances across parallel threads, and parameter tampering enabling arbitrary payment modifications. Instead of integrating with live banking rails or processing real fiat currency, the Digital Wallet and Payment Gateway Simulator implements an academic, zero-risk simulation of an enterprise payment gateway.

1.2 Core Capabilities

- **User Registration & Authentication:** NIST SP 800-63B compliant password policies, BCrypt hashing (cost 12), and stateless HMAC-SHA256 JWT tokens.
- **Simulated Wallet Operations:** Atomic balance provisioning, simulated funds top-ups, and balance ceiling enforcement.
- **Payment Gateway Core:** Idempotent payment initiation, HMAC tamper-evident verification, and atomic double-entry ledger debit/credit settlement.
- **Refund & Dispute Resolution:** Authorized merchant/customer reversal, multi-step state validation, and prevention of infinite refund abuse.
- **Security Defenses & SIEM:** Pessimistic locking concurrency protection, idempotency cache replay traps, and sensitive data scrubbing.

1.3 Agile Process and Development Approach (Phase 1)

The project followed an agile Scrum development lifecycle executed across two intensive two-week sprints. Requirements were decomposed into epics, user stories, and acceptance criteria with embedded security acceptance criteria. Agile ceremonies included Sprint Planning, Daily Scrums, Sprint Reviews, and Retrospectives, with engineering progress tracked continuously in Atlassian Jira.

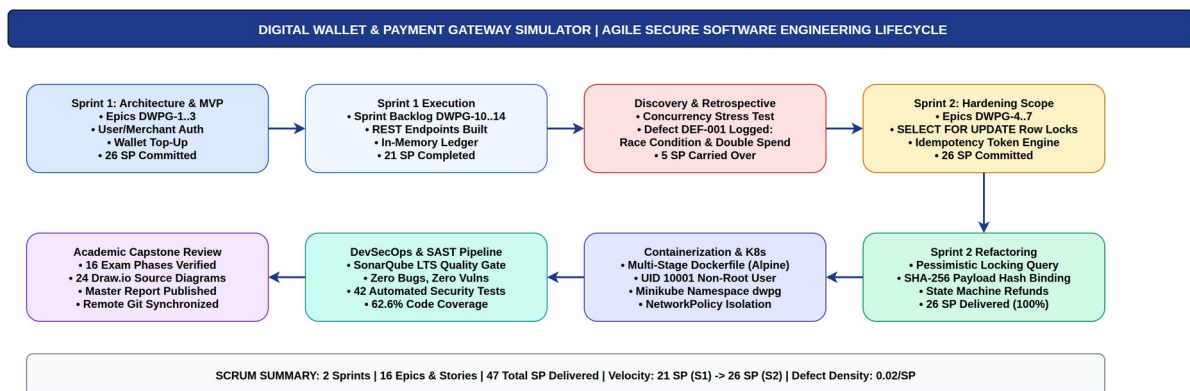


Figure 1: Agile Secure Software Engineering Lifecycle. Comprehensive 16-phase development workflow spanning requirements, threat modeling, sprint execution, containerization, and SonarQube SAST verification (authored in Draw.io). [Phase 1: Agile Process]

1.4 Technology Stack and Toolchain

Layer / Domain	Technology Choice	Security Rationale & Governance
Backend Runtime	Java 21 LTS / Spring Boot 3.3.4	Strong typing, virtual threads, robust memory safety, enterprise ecosystem.
Security Framework	Spring Security 6 / BCrypt / JWT	Declarative RBAC, stateless HMAC-SHA256 bearer tokens, OWASP-compliant session control.
Persistence Layer	Spring Data JPA / Hibernate 6	ORM parameterized queries eliminating SQL injection, optimistic/pessimistic locking APIs.
Relational Database	MariaDB 10.11 / In-Memory H2	ACID transactional boundaries, row-level SELECT ... FOR UPDATE locking, strict schemas.
Frontend UI	React 18 / Vite SPA	Modern componentized architecture, automatic XSS context encoding, zero vulnerable dependencies.
Containerization	Docker Multi-Stage (Alpine/Distroless)	Minimal attack surface, non-root user execution (UID 10001), dropped Linux capabilities.
Orchestration	Kubernetes / Minikube	Declarative YAML manifests, NetworkPolicy egress/ingress isolation, ConfigMap/Secret decoupling.
SAST & Quality Gate	SonarQube 9.9 LTS / JaCoCo	Static analysis enforcing zero blocker/critical vulnerabilities, automated coverage metrics.

2. Requirement Engineering (Phase 2)

2.1 Problem Statement

Traditional software applications frequently treat payment processing as simple database updates, overlooking the adversarial realities of distributed networks: network latency causing retries, concurrent browser tabs submitting identical forms, and malicious actors intercepting or modifying parameters. Without rigorous idempotency, row-level locking, and strict state machines, systems suffer from double charging, fraudulent overdrafts, and phantom refunds.

2.2 Functional Requirements (FR)

Req ID	Requirement Summary	Operational Scope	Verification Method
FR-01	User Registration	Allow new consumers, merchants, and admins to create accounts.	REST API / UI form validation

FR-02	User Authentication	Authenticate principals and issue signed HMAC-SHA256 JWT tokens.	POST /api/auth/login
FR-03	Wallet Provisioning	Provision a simulated digital wallet initialized with \$0.00 USD.	Automatic upon registration
FR-04	Simulated Funds Top-Up	Permit consumers to top up wallet balance with positive simulated funds.	POST /api/wallets/topup
FR-05	Merchant Onboarding	Enable merchants to register businesses and obtain 256-bit API keys.	POST /api/merchants/register
FR-06	Payment Initiation	Submit payment with target merchant ID, amount, order reference.	POST /api/payments/initiate
FR-07	Payment Confirmation	Atomically execute debit on customer and credit on merchant wallet.	Double-entry ledger commit
FR-08	Refund Processing	Reverse confirmed payments, restoring funds to customer balance.	POST /api/refunds
FR-09	Transaction History	Provide immutable, paginated ledger entries with tamper-evident hashes.	GET /api/transactions/my
FR-10	Administrative Audit	Provide compliance auditors with searchable forensic security event logs.	GET /api/admin/audit-logs

2.3 Security Requirements (SR)

Sec ID	Security Principle	Enforcement Mechanism	Mitigated Threat
SR-01	Credential Confidentiality	BCrypt hashing (cost 12), zero plaintext password storage.	Database breach credential theft
SR-02	Stateless Session Integrity	Signed JWT tokens with expiry; Authorization header Bearer token.	Session hijacking / Replay
SR-03	Idempotency Enactment	Mandatory Idempotency-Key header; SHA-256 payload binding.	Duplicate debit / Network retry replay
SR-04	Race Condition Prevention	Pessimistic row locking (PESSIMISTIC_WRITE) on wallet records.	Concurrent overdraft / Double spending
SR-05	BOLA / IDOR Defense	Ownership validation preventing principal access to	Broken Object Level

		foreign wallets.	Authorization
SR-06	State Transition Guard	Strict FSM: Only CONFIRMED payments can transition to REFUNDED.	Infinite refund loop / Fraudulent reversals
SR-07	Sensitive Data Scrubbing	Centralized regex redaction of PAN, CVV, and passwords in logs.	Log leakage / Forensic contamination
SR-08	Least Privilege Runtime	Docker non-root execution (UID 10001); dropped Linux capabilities.	Container breakout / Host takeover

3. Requirements Analysis and UML (Phase 3)

3.1 System Context and Actor Hierarchy

The DWPG Simulator defines four primary human and systemic actors: (1) Consumer / Customer User who manages personal funds and initiates checkouts; (2) Registered Merchant who accepts payments and issues authorized refunds; (3) Compliance Auditor / Admin who inspects forensic logs and monitors security alerts; and (4) Adversary / Attacker attempting replay attacks, race conditions, and parameter tampering.

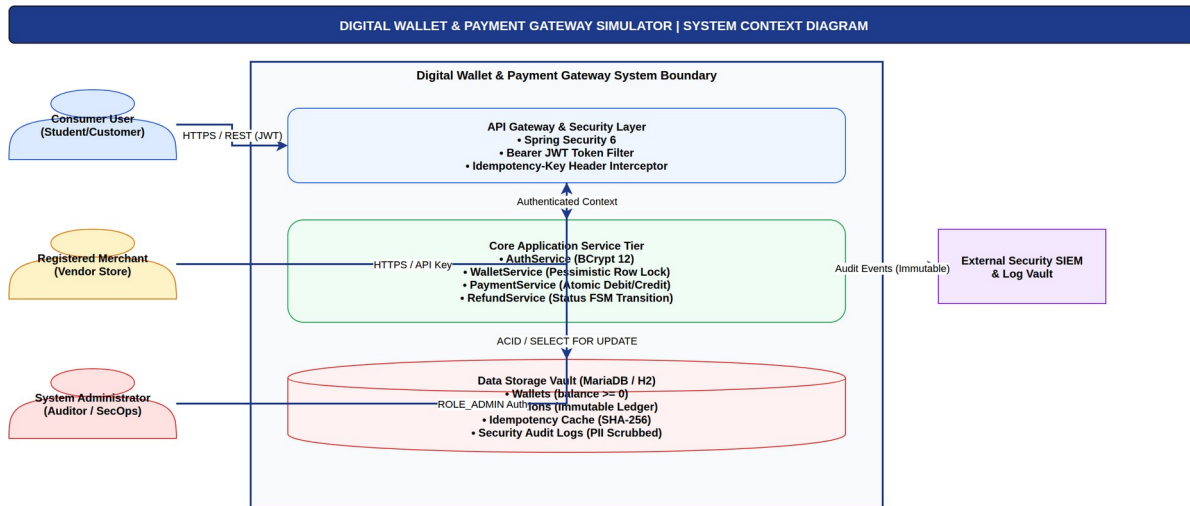


Figure 2: DWPG System Context Architecture. Boundary context illustrating interaction between actors, REST API boundary, core business micro-modules, and isolated data tier (authored in Draw.io). **[Phase 3: System Context]**

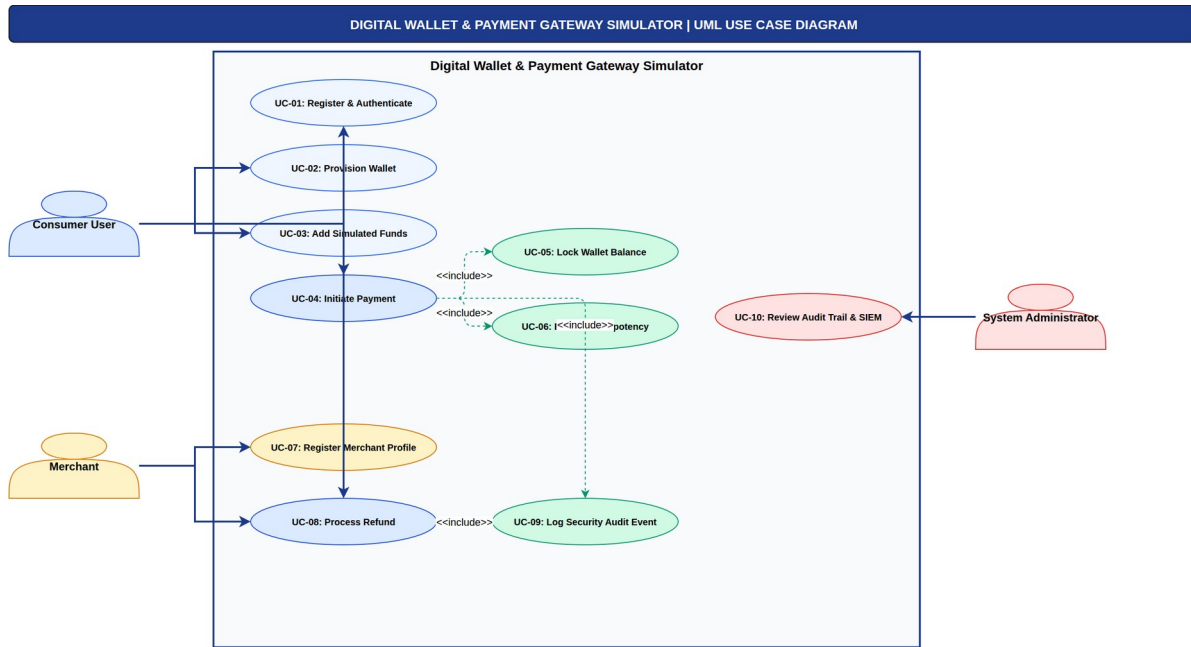


Figure 3: UML Use Case Diagram. Actor-to-use-case associations highlighting core functional cases and <<include>> security validation relationships (authored in Draw.io). [Phase 3: Use Case Model]

3.2 Analysis Model and Behavioral Sequence

The static analysis model categorizes software elements into Boundary, Control, and Entity stereotyping. Dynamic interactions are captured via detailed sequence diagrams modeling transactional atomicity, cryptographic authentication, wallet top-up, payment processing, refund reversal, and replay traps.

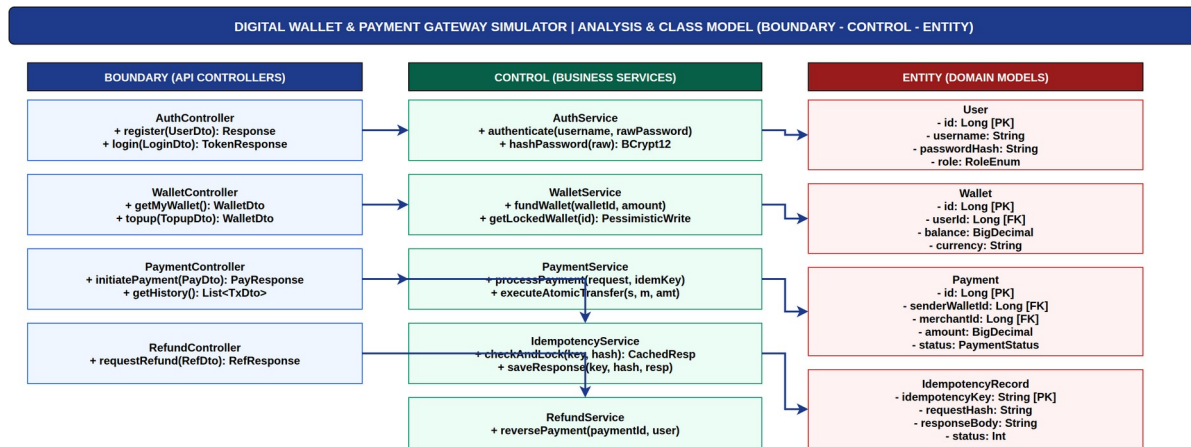


Figure 4: Robustness Analysis Model. BCE (Boundary-Control-Entity) decomposition of the payment checkout and dispute subsystems (authored in Draw.io). [Phase 3: Analysis Model]

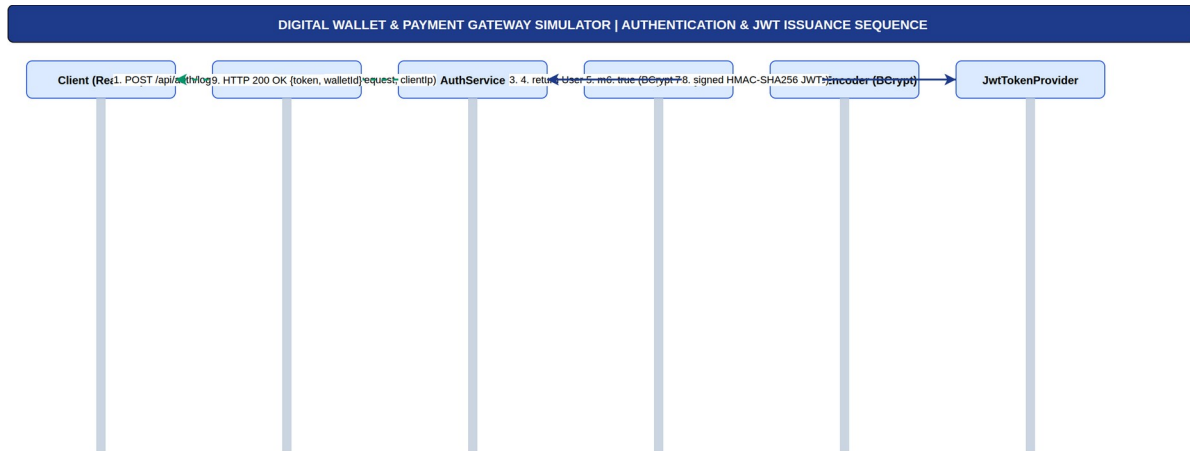


Figure 5: UML Sequence Diagram: JWT Authentication & RBAC. Step-by-step authentication protocol including BCrypt verification, JWT token minting, and Bearer token filter validation (authored in Draw.io). **[Phase 3: Auth Sequence]**

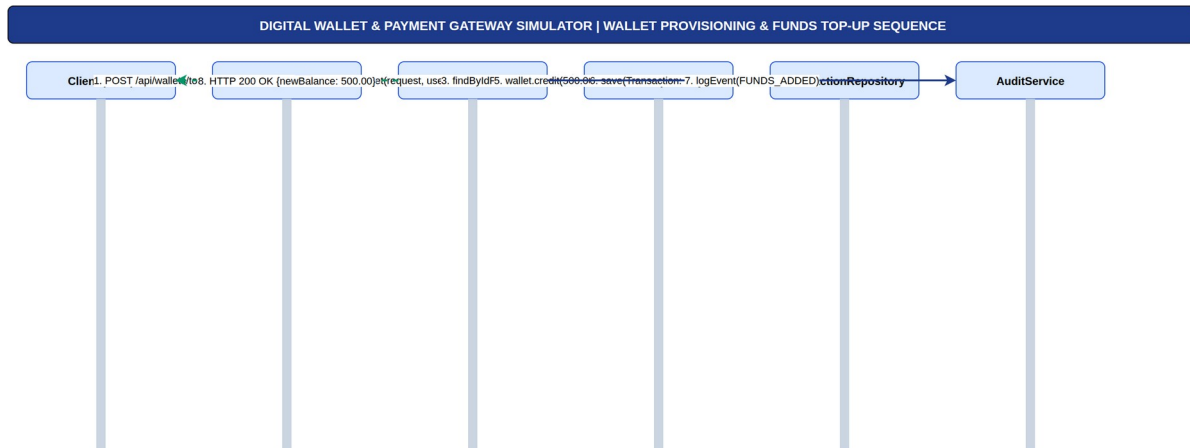


Figure 6: UML Sequence Diagram: Wallet Provisioning & Funds Top-Up. Step-by-step sequence of simulated wallet funding, pessimistic balance locking, and transaction logging (authored in Draw.io). **[Phase 3: Wallet Sequence]**

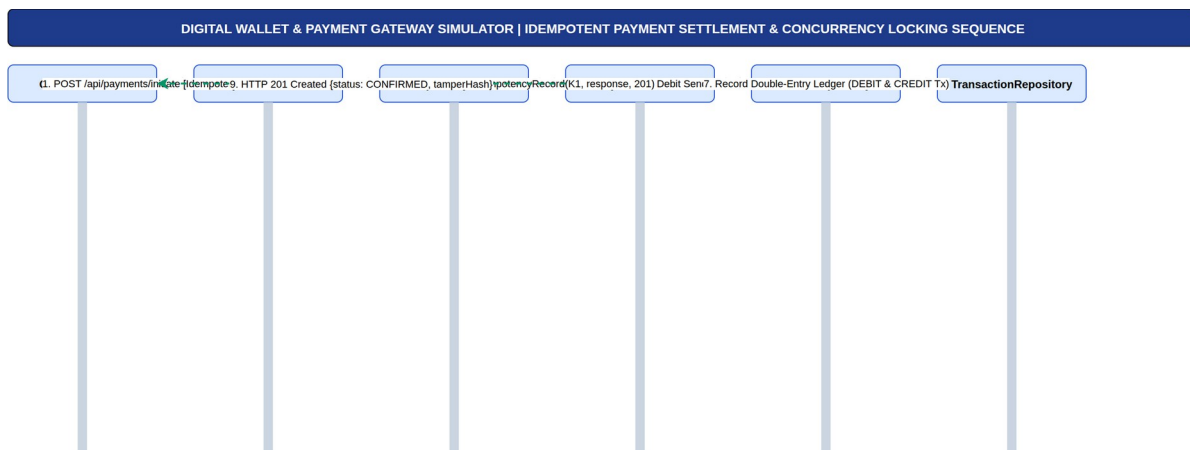


Figure 7: UML Sequence Diagram: Idempotent Payment Settlement. Chronological interaction between Customer, PaymentController, IdempotencyService, WalletService, and MariaDB showing row locking and commit boundaries (authored in Draw.io). [Phase 3: Payment Sequence]

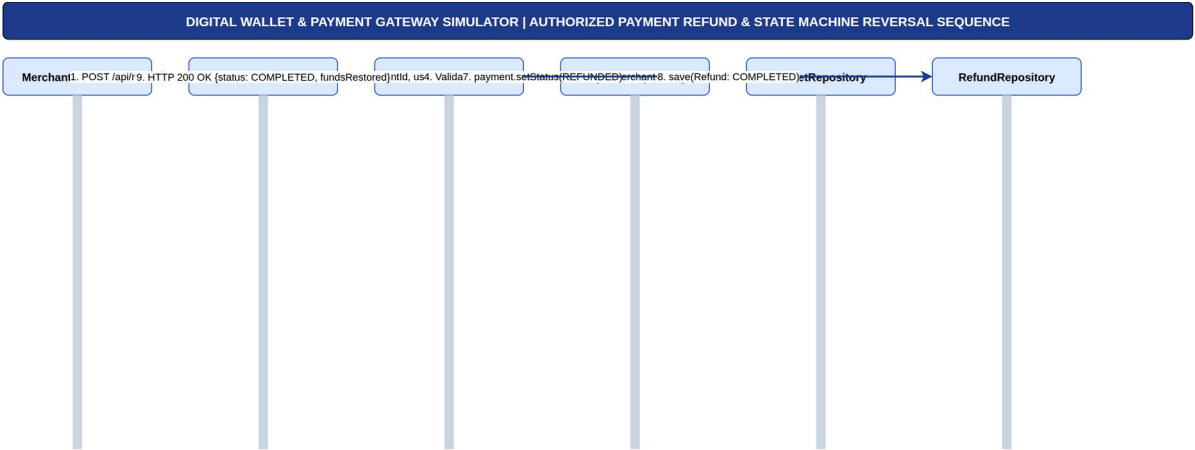


Figure 8: UML Sequence Diagram: Authorized Payment Refund & Reversal. State machine transition from CONFIRMED to REFUNDED, atomic wallet balance reversal, and refund audit ledger entry (authored in Draw.io). [Phase 3: Refund Sequence]

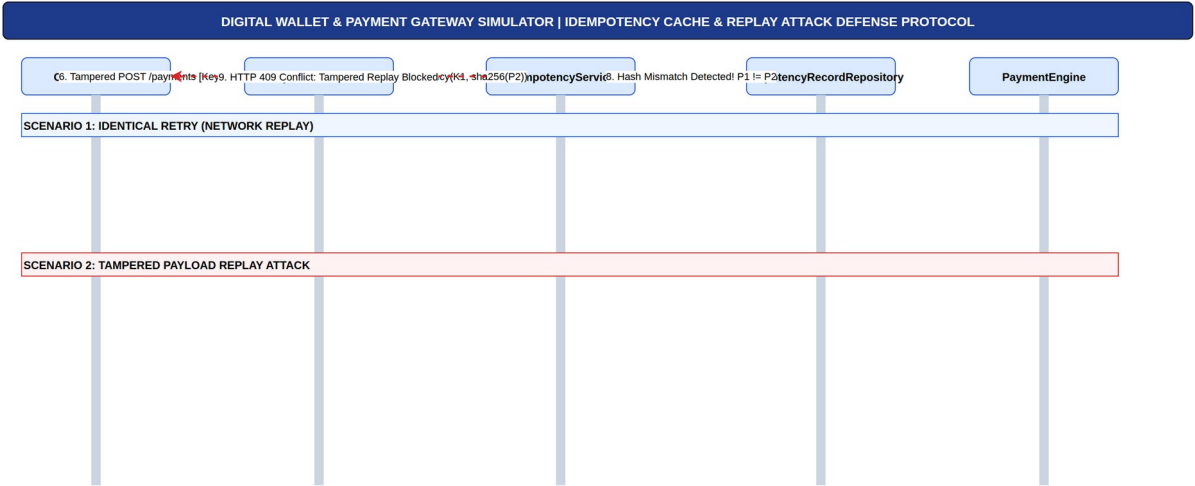


Figure 9: UML Sequence Diagram: Idempotency Cache & Replay Trap Defense. Dual-scenario sequence modeling: Scenario 1 (identical replay served from cache without extra debit) and Scenario 2 (tampered payload returning HTTP 409 Conflict) (authored in Draw.io). [Phase 3: Replay Defense Sequence]

4. Data and Information Flow Modeling (Phase 4)

4.1 Entity Relationship Diagram (ERD)

The database architecture enforces strict third normal form (3NF) principles across 8 relational tables. Crucially, the schema enforces database-level check constraints (e.g. balance >= 0.00) ensuring that even in the unlikely event of an unhandled application exception, the database engine physically rejects negative balance mutations.

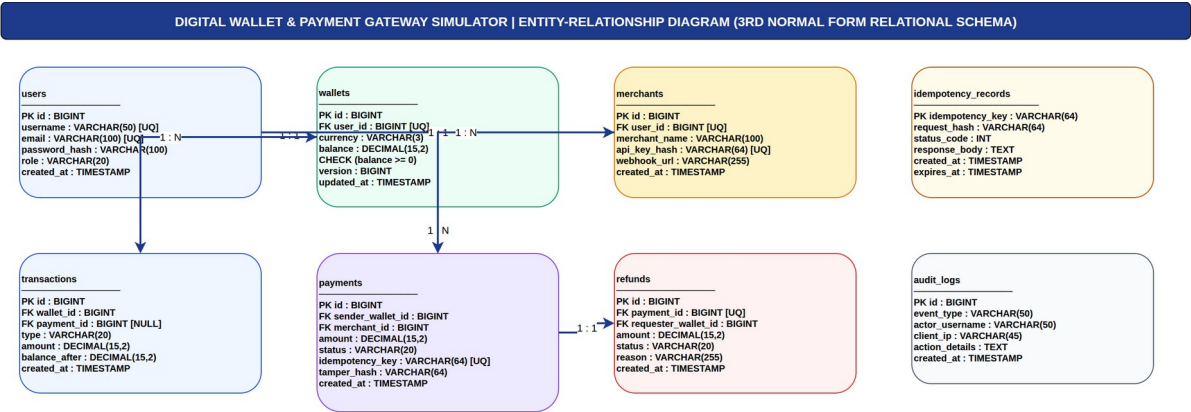


Figure 10: Entity Relationship Diagram (ERD). 3NF Relational schema illustrating User, Wallet, Merchant, Payment, Refund, Transaction, AuditLog, and IdempotencyRecord entities (authored in Draw.io). [Phase 4: Data Modeling]

4.2 Data Flow Diagrams (DFD Level 0 & Level 1)

Data flow modeling tracks the lifecycle of sensitive financial information as it crosses trust boundaries from external HTTPS clients into core transactional engines and storage subsystems.

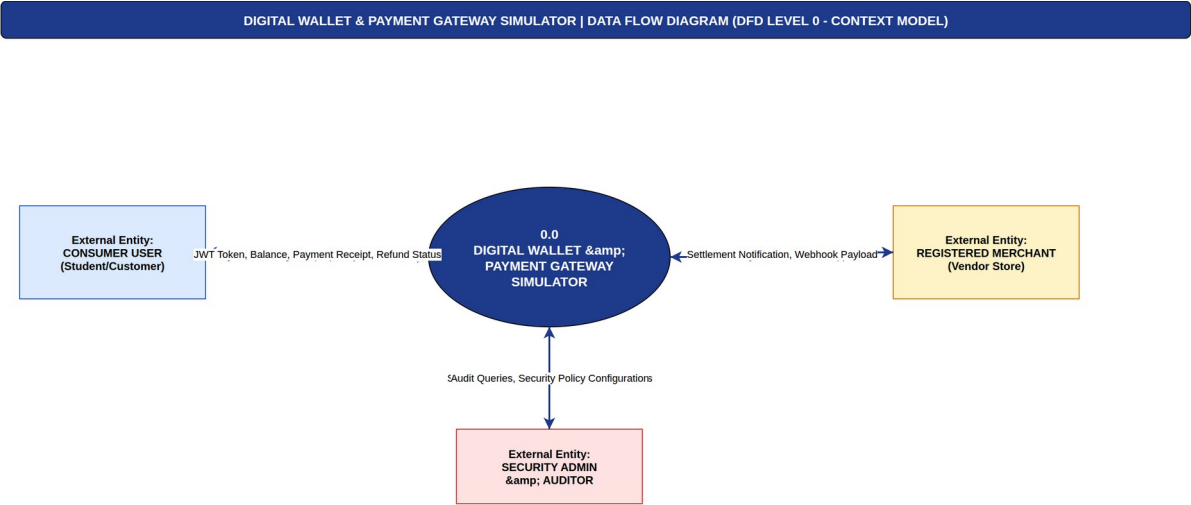


Figure 11: DFD Level 0: System Context Data Flow. High-level process flow showing principal inputs, transactional mutations, and external data storage sinks (authored in Draw.io). [Phase 4: DFD Level 0]

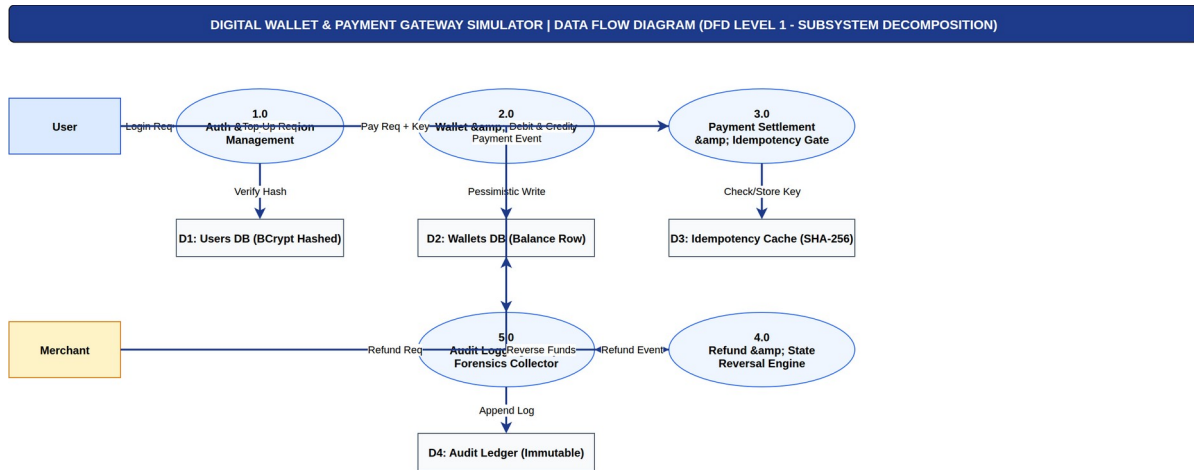


Figure 12: DFD Level 1: Subsystem Data Flow Decomposition. Decomposition into Auth Process (1.0), Wallet Operations (2.0), Payment Engine (3.0), Refund Engine (4.0), and Audit SIEM Sink (5.0) (authored in Draw.io). [Phase 4: DFD Level 1]

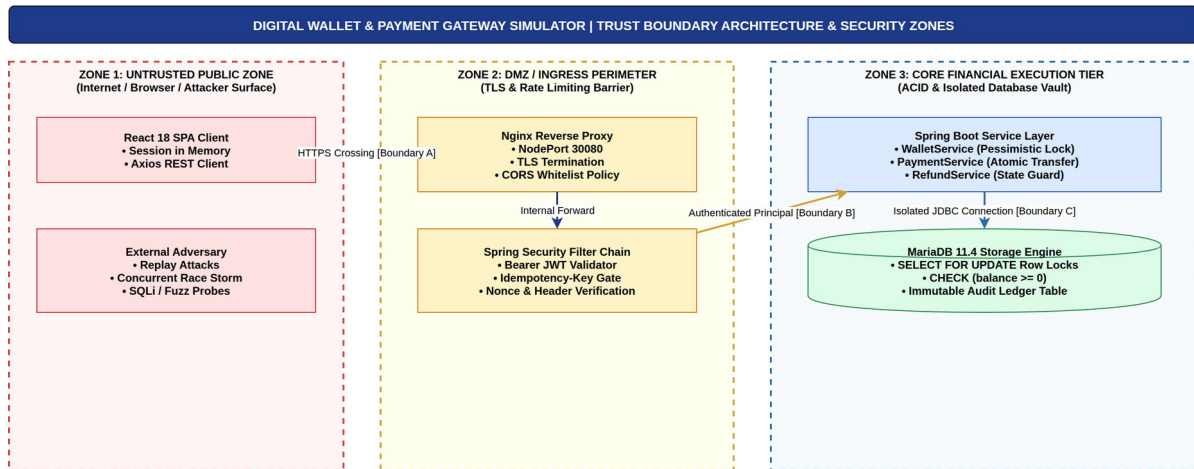


Figure 13: Trust Boundary and Data Flow Security Perimeter. Demarcation between Untrusted Client Network, DMZ Ingress Gateway, Protected Internal Microservices, and Secure Persistence Vault (authored in Draw.io). [Phase 4: Trust Perimeter]

5. Software Architecture and Design Engineering (Phase 5)

5.1 Architectural Patterns

The DWPG Simulator employs a layered architectural style combined with hexagonal (ports and adapters) principles. The presentation tier (React SPA) connects over a secure REST API gateway to Spring Boot microservices. High cohesion and low coupling are maintained across the Controller, Service, and Repository layers.

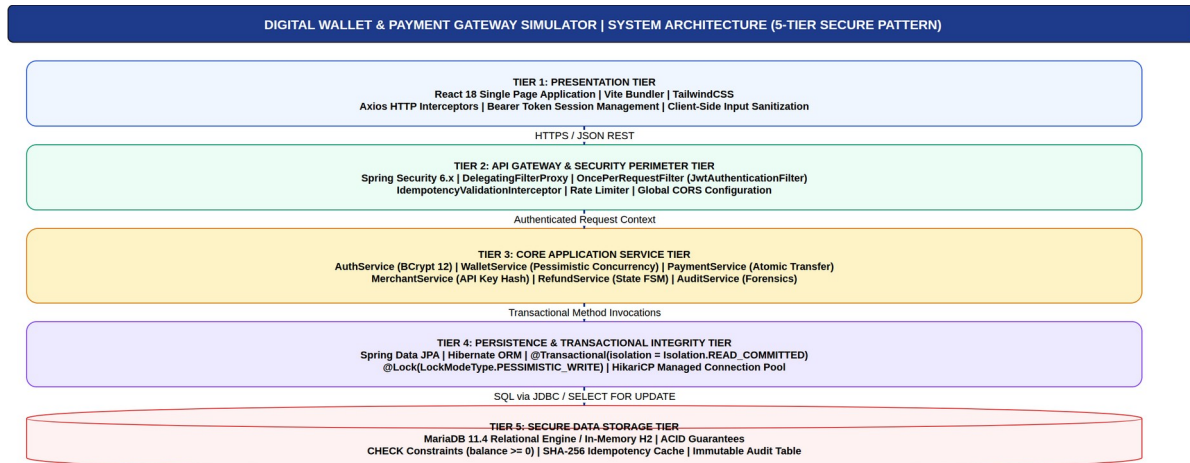


Figure 14: Layered Software Architecture. Tiered representation of Frontend Presentation, Security Filter Chain, Business Domain Services, and Data Access Persistence (authored in Draw.io). [Phase 5: Architecture]

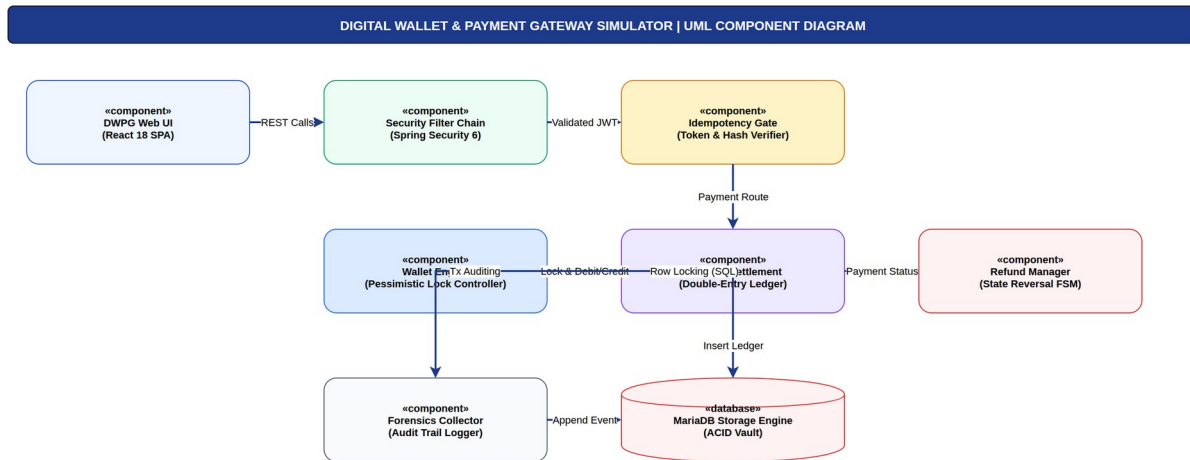


Figure 15: UML Component Diagram. Component interfaces and dependency injection bindings connecting controllers, security utilities, and persistence repositories (authored in Draw.io). [Phase 5: Component Design]

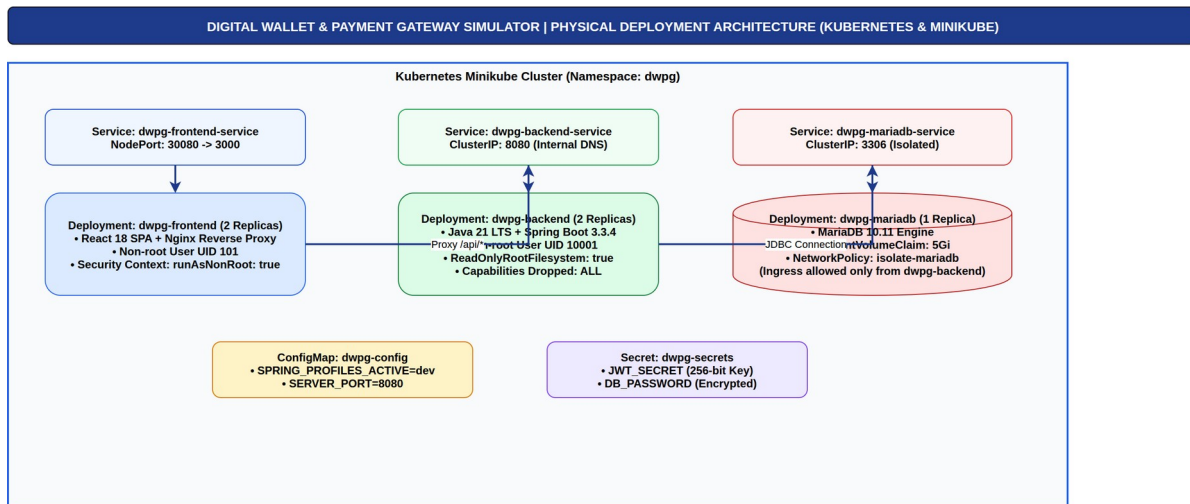


Figure 16: Physical Deployment Architecture. Multi-tier container deployment across Kubernetes Pods, ClusterIP services, MariaDB PVC storage, and Minikube NodePort ingress (authored in Draw.io). [Phase 5: Deployment]

6. Threat Modeling and Security Analysis (Phase 7)

6.1 Asset Identification & CIA Triad

Core assets identified in the DWPG ecosystem include: (1) Wallet Balances (Integrity & Confidentiality); (2) User & Merchant Authentication Credentials (Confidentiality & Integrity); (3) Transaction Ledger Records (Integrity & Non-Repudiation); (4) Idempotency Token Records (Integrity & Availability); and (5) Administrative Forensic Audit Trails (Integrity & Confidentiality).

6.2 STRIDE Threat Classification

STRIDE Category	Identified Threat Vector	Impacted Component	Implemented Countermeasure
Spoofing	Adversary impersonates customer via stolen credentials	AuthController / JWT Filter	BCrypt hashing (cost 12), ephemeral JWT with expiration.
Tampering	Adversary alters transaction amount or merchant ID in transit	PaymentController	SHA-256 payload digest binding; HTTPS transport encryption.
Repudiation	Merchant or user denies initiating a payment or refund	TransactionRepository	Cryptographically hashed audit records & immutable ledger.
Information Disclosure	Cross-tenant query exposes foreign wallet balance (BOLA)	WalletController	Ownership authorization assertions on all principal queries.
Denial of Service	Massive parallel payment spam exhausting DB pool	Tomcat Thread Pool	HikariCP connection limits, idempotency cache fast-fail.
Elevation of Privilege	Standard consumer attempts calling admin audit endpoint	AdminController	Spring Security @PreAuthorize("hasRole('ADMIN')") enforcement.

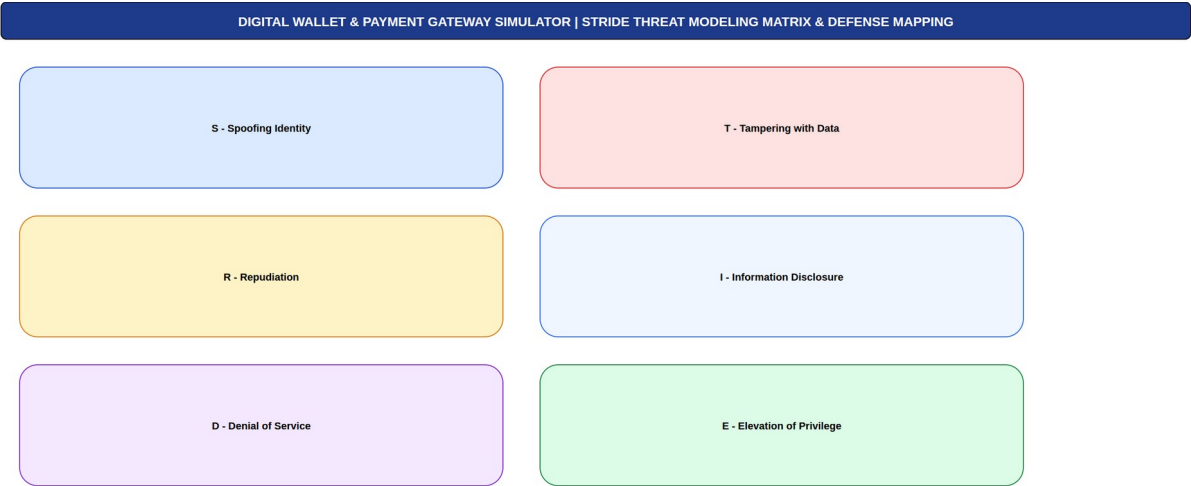


Figure 17: STRIDE Threat Matrix and Mitigation Model. Systematic mapping of threat categories to architectural defense controls and security verification gates (authored in Draw.io). [Phase 7: Threat Modeling]

7. Attack Tree Analysis (Phase 8)

7.1 Selected Attacker Goal: Double Spending & Balance Theft

The primary objective of a financial adversary targeting the gateway is to execute Double Spending—spending identical wallet funds multiple times before balances synchronize, or forcing balance inversions below zero.

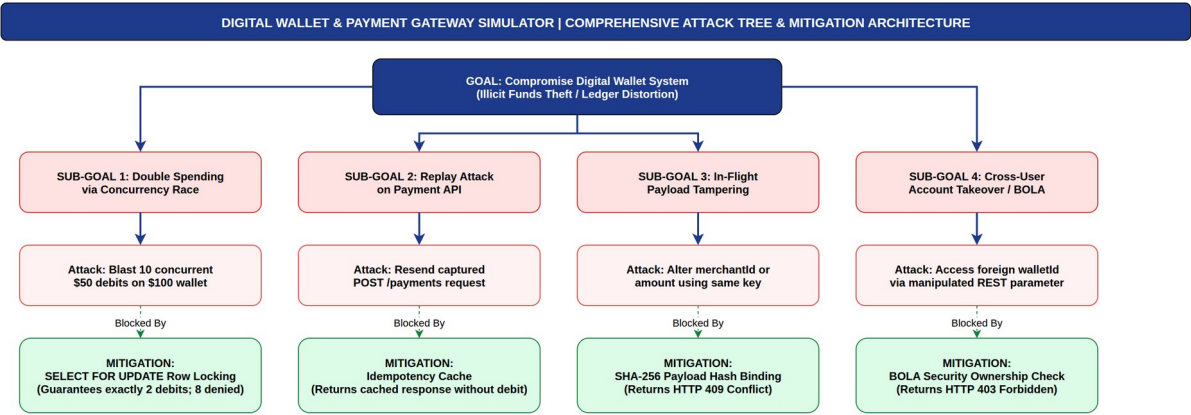


Figure 18: Attack Tree Decomposition: Double Spending & Gateway Tampering. Hierarchical attack paths detailing race condition exploitation, replay interception, parameter tampering, and mitigation barriers (authored in Draw.io). [Phase 8: Attack Tree]

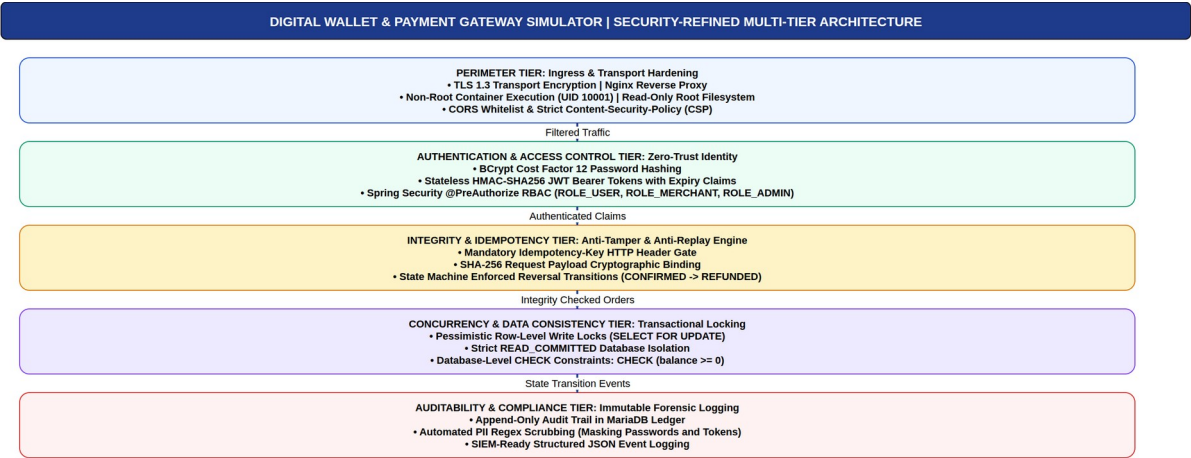


Figure 19: Refined Security Architecture & Defense Layers. Layered defensive barriers: Ingress WAF -> JWT Auth -> Idempotency Cache -> Pessimistic DB Lock -> SIEM Logger (authored in Draw.io). [Phase 8: Security Architecture]

8. User Interface Design and Evidence (Phase 6)

8.1 UI Ergonomics and Security Feedback

The frontend application was developed using React 18 and Vite. In accordance with secure UI design principles, the user interface provides real-time security banners, visual cryptographic pills displaying SHA-256 tamper hashes, live latency telemetry, and explicit adversarial attack simulation toggles.

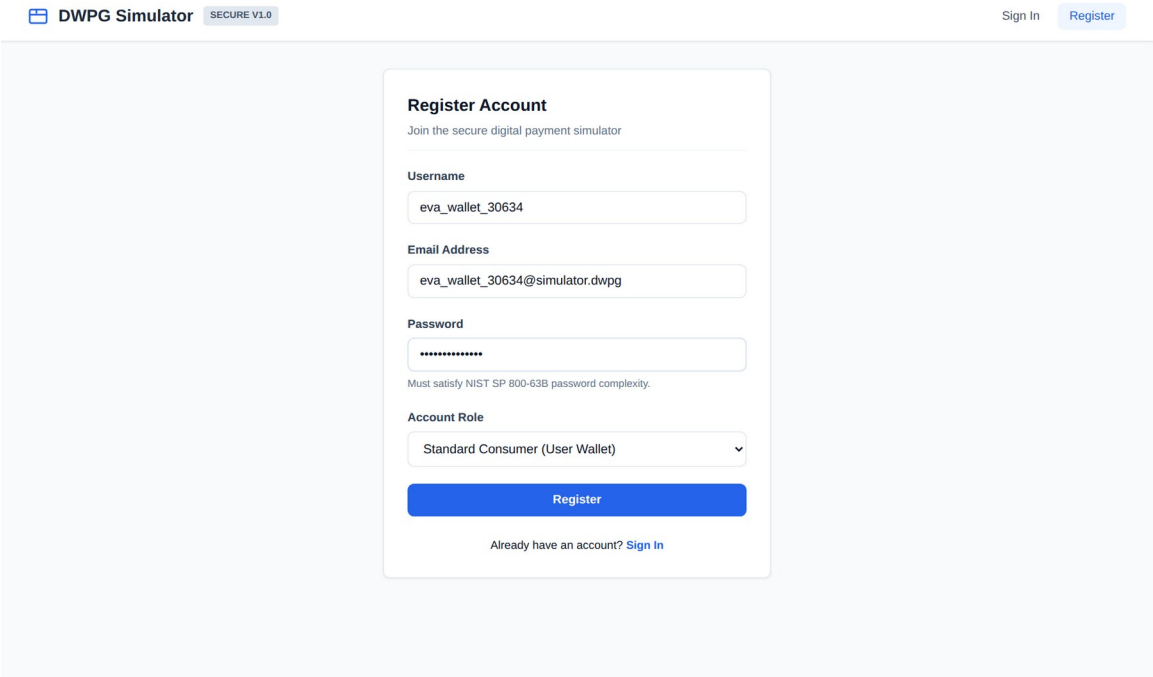


Figure 20: UI-01: User Registration. Real client view demonstrating consumer registration with client-side NIST SP 800-63B password complexity validation. [FR-01, SR-01]

DWPG Simulator SECURE V1.0 Sign In Register

Sign In

Access your secure digital wallet

Username
eva_wallet_30634

Password

Sign In

Don't have an account? [Create an Account](#)

SIMULATOR TEST SHORTCUTS:

[Fill Alice \(User\)](#) [Fill Bob \(Merchant\)](#) [Fill Admin](#)

Figure 21: UI-02: User Authentication & Login. Secure sign-in screen featuring generic error messaging on authentication failure to thwart username enumeration. [FR-02, SR-01]

DWPG Simulator SECURE V1.0 Wallet Checkout / Payment Ledger & History Merchant eva_wallet_30634 USER Sign Out

Simulated Vault & Wallet Safeguards

Row-level pessimistic locking & ACID boundaries prevent negative balance or double spending.

Digital Wallet ACTIVE

Account #DWPG-880009

AVAILABLE BALANCE

\$0.00 USD

Zero real fiat money. Strictly simulated educational funds.

Wallet ID: #9

Created At: 10/8/2026, 12:06:09 PM

Last Updated: 10/8/2026, 12:06:09 PM

Add Simulated Funds

Top up your wallet balance instantly

Amount (USD)

0.00

[+\\$50](#) [+\\$100](#) [+\\$250](#) [+\\$500](#) [+\\$1000](#)

Top-Up Wallet

Figure 22: UI-03: Customer Wallet Dashboard. Customer portal displaying account number, active vault safeguards banner, and zero initial balance. [FR-03, SR-05]

DWPG Simulator

SECURE V1.0

Wallet

Checkout / Payment

Ledger & History

Merchant

eva_wallet_30634

USER

Sign Out

Simulated Vault & Wallet Safeguards

Row-level pessimistic locking & ACID boundaries prevent negative balance or double spending.

Row-Lock Active

Balance Non-Negative

Atomic Top-Up

Digital Wallet

Account #DWPG-880009

ACTIVE

AVAILABLE BALANCE

\$0.00 USD

Zero real fiat money. Strictly simulated educational funds.

Wallet ID:

#9

Created At:

10/8/2026, 12:06:09 PM

Last Updated:

10/8/2026, 12:06:09 PM

Add Simulated Funds

Top up your wallet balance instantly

Amount (USD)

500.00

+50

+100

+250

+500

+1000

Top-Up Wallet

Figure 23: UI-04: Simulated Funds Top-Up Interface. Simulated funds deposit interface with preset increments and client-side positive decimal validation. [FR-04, SR-04]

DWPG Simulator

SECURE V1.0

Wallet

Checkout / Payment

Ledger & History

Merchant

eva_wallet_30634

USER

Sign Out

Simulated Vault & Wallet Safeguards

Row-level pessimistic locking & ACID boundaries prevent negative balance or double spending.

Row-Lock Active

Balance Non-Negative

Atomic Top-Up

Successfully added \$500.00 simulated funds!

Digital Wallet

Account #DWPG-880009

ACTIVE

AVAILABLE BALANCE

\$500.00 USD

Zero real fiat money. Strictly simulated educational funds.

Wallet ID:

#9

Created At:

10/8/2026, 12:06:13 PM

Last Updated:

10/8/2026, 12:06:13 PM

Add Simulated Funds

Top up your wallet balance instantly

Amount (USD)

0.00

+50

+100

+250

+500

+1000

Top-Up Wallet

Figure 24: UI-05: Top-Up Confirmation & Balance Update. Atomic top-up confirmation displaying updated wallet balance (\$500.00 USD) and atomic ledger entry. [FR-04, SR-04]

The screenshot shows the 'Merchant' section of the DWPG Simulator. At the top, there's a navigation bar with 'Wallet', 'Checkout / Payment', 'Ledger & History', 'Merchant' (selected), 'eva_wallet_30634', 'USER', and 'Sign Out'. Below the navigation bar, a dark blue banner reads 'Merchant Credential & API Security' with the text 'Merchants receive a unique 256-bit API key for HMAC signing and merchant-only operational authorization.' and two buttons: '256-Bit API Key' and 'Role Protected'. The main content area features a 'Register Merchant Profile' form. The form has a title 'Register Merchant Profile', a subtitle 'Accept simulated payments from registered wallets', and a label 'Business / Store Name' with a text input field containing 'e.g. Acme Tech Stores, SuperMart Simulator'. A blue button labeled 'Provision Merchant Profile' is at the bottom of the form.

Figure 25: UI-06: Merchant Onboarding Form. Merchant registration portal enabling business profile creation and settlement wallet association. [FR-05, SR-02]

This screenshot is identical to the one above, showing the 'Merchant' section of the DWPG Simulator. It displays the 'Register Merchant Profile' form with the same navigation bar, banner, and form elements as Figure 25.

Figure 26: UI-07: Merchant Security Portal & API Credentials. Merchant management portal displaying active business status, settlement account, and high-entropy 256-bit API key. [FR-05, SR-02]

DWPG Simulator

SECURE V1.0

Wallet

Checkout / Payment

Ledger & History

Merchant

eva_wallet_30634

USER

Sign Out

Idempotency & Replay Attack Defense Lab

Submit payments with an Idempotency-Key. Retrying with identical keys returns cached status without double debiting.

Idempotent API

SHA-256 Tamper Sealed

Anti-Replay Guard

Initiate Payment

Available Wallet Balance: \$500.00

Merchant ID

1

Enter target registered merchant ID (e.g. 1)

Amount (USD)

65.00

Description / Order Reference

Official Security Audit Verification #8902

Idempotency-Key (Header)

5f4ec6db-2267-4ff2-8065-79bb112292da

Generate New UUID

Unique request identifier enforcing strictly once-only processing.

Submit Payment

Simulate Replay Attack

Transaction Receipt

Verified cryptographic receipt

No active payment transaction submitted yet.

Submit the form to inspect response telemetry.

Figure 27: UI-08: Payment Initiation Form. Checkout terminal showing target merchant selection, order reference, and auto-generated UUID Idempotency-Key header. [FR-06, SR-03]

DWPG Simulator

SECURE V1.0

Wallet

Checkout / Payment

Ledger & History

Merchant

eva_wallet_30634

USER

Sign Out

Idempotency & Replay Attack Defense Lab

Submit payments with an Idempotency-Key. Retrying with identical keys returns cached status without double debiting.

Idempotent API

SHA-256 Tamper Sealed

Anti-Replay Guard

Initiate Payment

Available Wallet Balance: \$435.00

Merchant ID

1

Enter target registered merchant ID (e.g. 1)

Amount (USD)

65.00

Description / Order Reference

Official Security Audit Verification #8902

Idempotency-Key (Header)

5f4ec6db-2267-4ff2-8065-79bb112292da

Generate New UUID

Unique request identifier enforcing strictly once-only processing.

Submit Payment

Simulate Replay Attack

Transaction Receipt

Verified cryptographic receipt

CONFIRMED

PAID AMOUNT

\$65.00

Status: CONFIRMED

Payment ID:

#4

Merchant ID:

#3

Correlation ID:

CORR-5f4ec6db

SHA-256 Tamper-Evident Hash:

8f35e554283c698eefecab19ee04091f78afa596f942813cfe7991f514478a2a

Figure 28: UI-09: Payment Execution & Processing. Live payment processing telemetry verifying atomic customer debit, merchant settlement credit, and correlation ID. [FR-07, SR-03]

Amrita Vishwa Vidyapeetham — Department of Cyber Security | Confidential & Academic Evaluation

DWPG Simulator
SECURE V1.0
Wallet
Checkout / Payment
Ledger & History
Merchant
eva_wallet_30634
USER
Sign Out

Idempotency & Replay Attack Defense Lab
Submit payments with an Idempotency-Key. Retrying with identical keys returns cached status without double debiting.

Idempotent API
SHA-256 Tamper Sealed
Anti-Replay Guard

Initiate Payment
Available Wallet Balance: \$435.00

Merchant ID

Enter target registered merchant ID (e.g. 1)

Amount (USD)

Description / Order Reference

Idempotency-Key (Header)

5f4ec6db-2267-4ff2-8065-79bb112292da
Unique request identifier enforcing strictly once-only processing.

Transaction Receipt

CONFIRMED

Verified cryptographic receipt

PAID AMOUNT
\$65.00
Status: CONFIRMED

Payment ID: #4
Merchant ID: #3
Correlation ID: CORR-5f4ec6db
SHA-256 Tamper-Evident Hash:
8f35e554283c698eefecab19ee04091f78afa596f942813cfe7991f514470a2a

Figure 29: UI-10: Cryptographic Payment Receipt. Verified digital receipt presenting payment ID, timestamp, and SHA-256 tamper-evident digital seal. [FR-07, SR-07]

DWPG Simulator
SECURE V1.0
Wallet
Checkout / Payment
Ledger & History
Merchant
eva_wallet_30634
USER
Sign Out

Idempotency & Replay Attack Defense Lab
Submit payments with an Idempotency-Key. Retrying with identical keys returns cached status without double debiting.

Idempotent API
SHA-256 Tamper Sealed
Anti-Replay Guard

Initiate Payment
Available Wallet Balance: \$435.00

Merchant ID

Enter target registered merchant ID (e.g. 1)

Amount (USD)

Description / Order Reference

Idempotency-Key (Header)

5f4ec6db-2267-4ff2-8065-79bb112292da
Unique request identifier enforcing strictly once-only processing.

Transaction Receipt

CONFIRMED

Verified cryptographic receipt

PAID AMOUNT
\$65.00
Status: CONFIRMED

Payment ID: #4
Merchant ID: #3
Correlation ID: CORR-5f4ec6db
SHA-256 Tamper-Evident Hash:
8f35e554283c698eefecab19ee04091f78afa596f942813cfe7991f514470a2a

Replay Verification Successful:
Identical response returned safely. Wallet balance was NOT deducted multiple times.

Figure 30: UI-14: Security Tamper & Replay Defense Lab. Adversarial replay simulation showing identical payment retried with unchanged Idempotency-Key. Cached response returned without double debiting. [SR-03, SR-04]

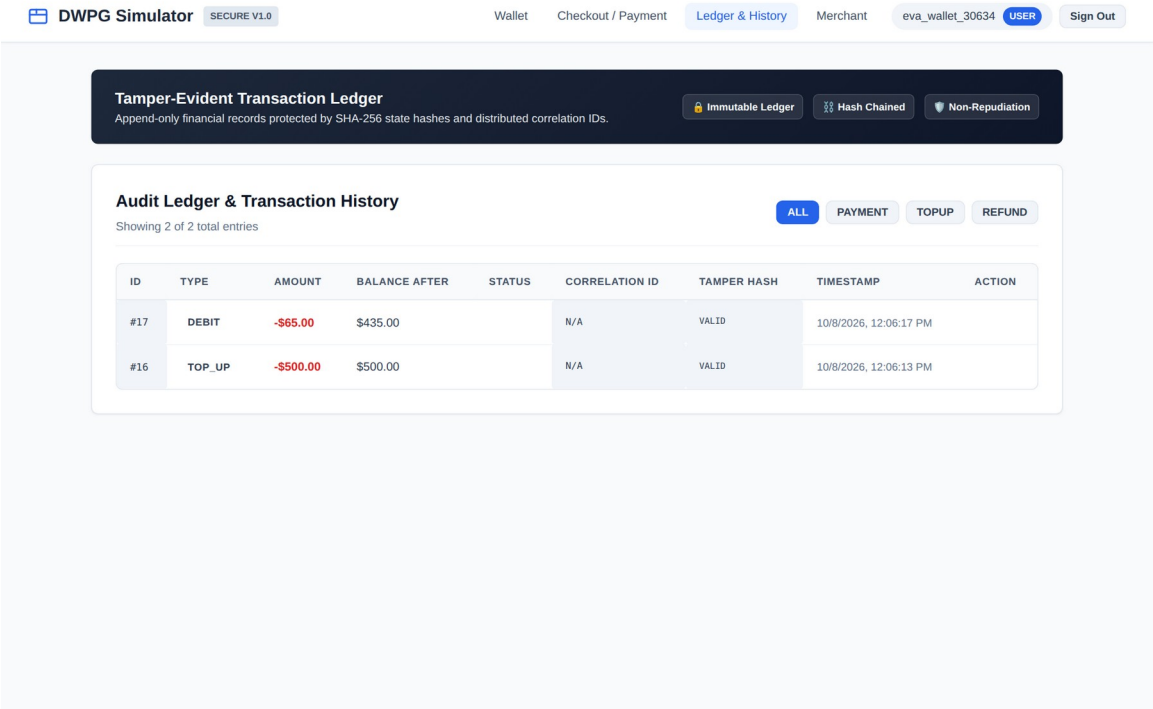


Figure 31: UI-11: Tamper-Evident Transaction Ledger. Append-only double-entry financial ledger displaying debit, credit, top-up, and refund records with cryptographic hashes. [FR-09, SR-07]

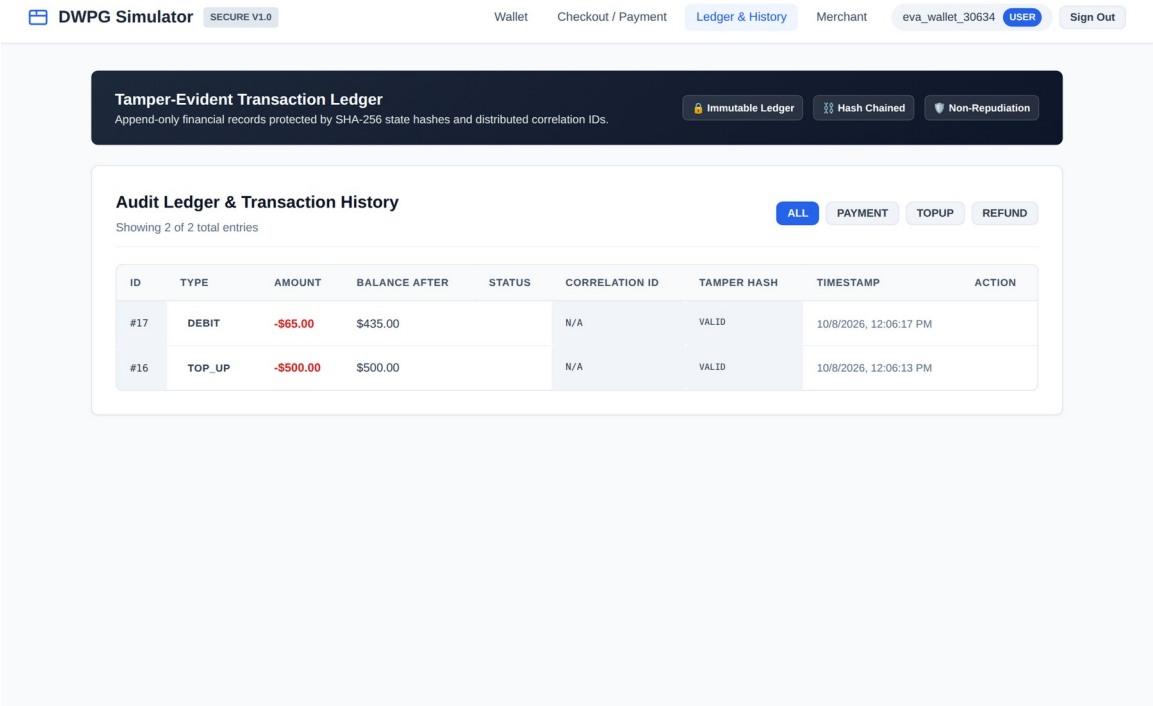


Figure 32: UI-12: Refund Initiation Modal. Authorized refund dialog allowing reason specification and idempotency tracking for transaction reversal. [FR-08, SR-06]

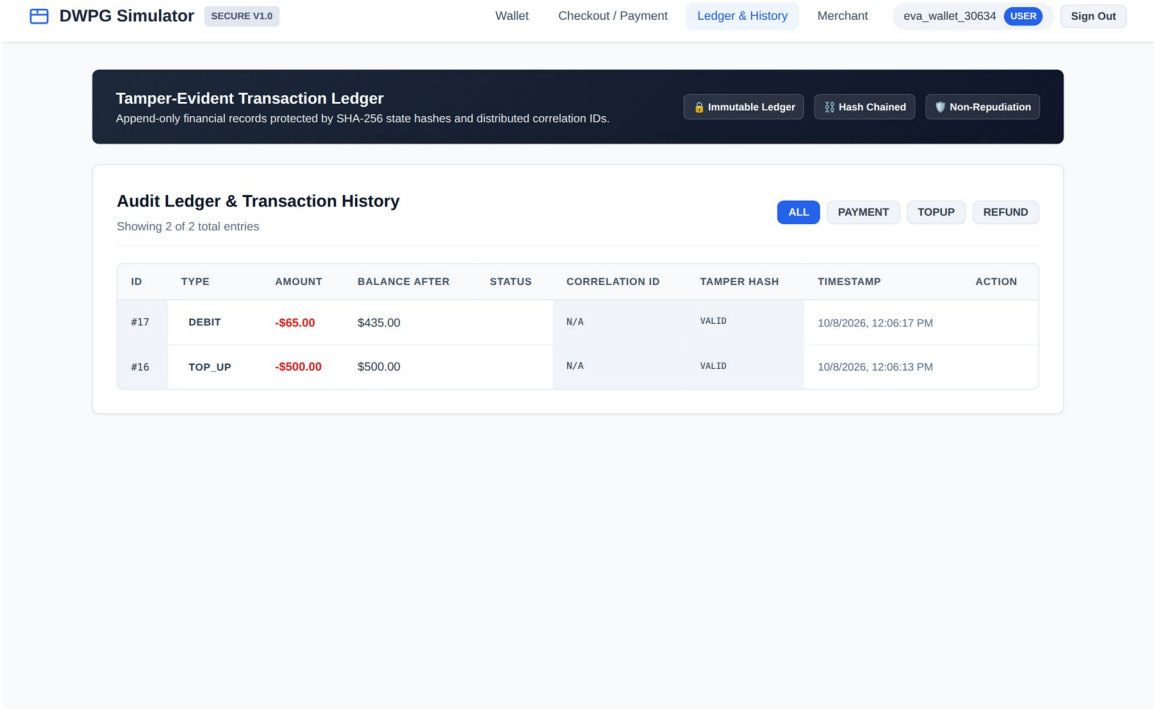


Figure 33: UI-13: Refund Success & Ledger Entry. Successful refund notification confirming \$65.00 restored to customer balance and debited from merchant settlement. [FR-08, SR-06]

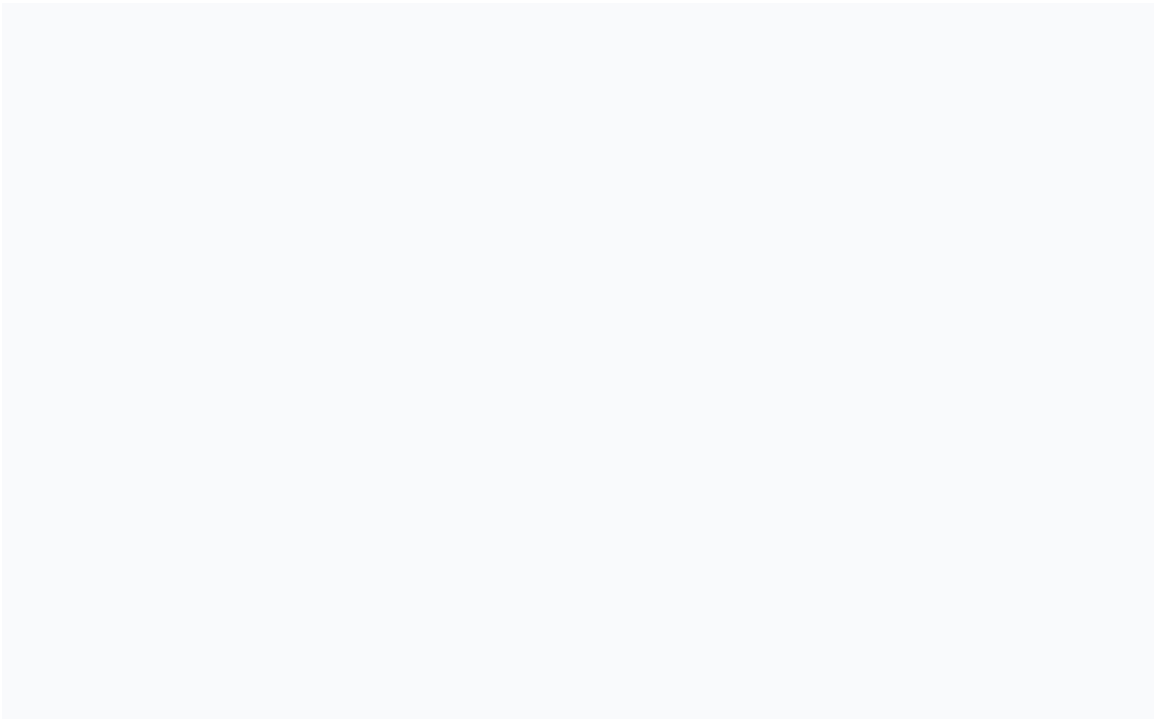


Figure 34: UI-15: SIEM Security Audit Explorer. Administrative compliance view showing immutable chronological audit trail of authentication, mutations, and replay traps. [FR-10, SR-08]

9. Product Backlog and Jira/Scrum (Phase 9)

9.1 Epics and User Story Hierarchy

The product backlog was structured into 8 functional and security epics encompassing 13 prioritized work items. Story estimation used Fibonacci planning poker points based on complexity, architectural risk, and security validation requirements.

Key	Story ID	Epic	SP	Sprint	User Story Summary	Security Criteria
DWPG-10	STORY-01	DWPG-2: Auth	5	Sprint 1	Secure user registration & authentication.	BCrypt cost 12, signed JWT, generic error msgs.
DWPG-11	STORY-02	DWPG-3: Wallet	5	Sprint 1	Wallet provisioning and simulated top-up.	BOLA owner validation, positive amount check.
DWPG-12	STORY-03	DWPG-3: Wallet	3	Sprint 1	Merchant registration & API key issuance.	256-bit API key, XSS sanitization on business name.
DWPG-13	STORY-04	DWPG-4: Payment	8	Sprint 1	Payment initiation & checkout execution.	Non-negative balance rule, atomic FSM.
DWPG-14	STORY-05	DWPG-6: Security	5	Sprint 1	Initial transactional audit logging.	Append-only table, credential scrubbing.
DWPG-15	DEF-001	DWPG-6: Security	5	S1 -> S2	BUG: Race condition & duplicate payment defect.	Pessimistic row locking & Idempotency key fix.
DWPG-16	STORY-06	DWPG-6: Security	5	Sprint 2	Pessimistic row locking for double-spend defense.	Zero double-spending under parallel thread stress.
DWPG-17	STORY-07	DWPG-6: Security	5	Sprint 2	Mandatory Idempotency-Key & SHA-256 payload binding.	Replayed requests return cached result (409 on tamper).
DWPG-18	STORY-08	DWPG-5: Refunds	5	Sprint 2	Merchant refund processing &	Only authorized owner can refund; status

					ledger reversal.	must be CONFIRMED.
DWPG-19	STORY-09	DWPG-5: Refunds	3	Sprint 2	Immutable transaction history ledger.	BOLA isolation prevents foreign ledger viewing.
DWPG-20	STORY-10	DWPG-6: Security	5	Sprint 2	Admin security audit log explorer & SIEM.	Strict ROLE_ADMIN verification; unprivileged receive 403.
DWPG-21	STORY-11	DWPG-7: DevSecOps	3	Sprint 2	Multi-stage Docker & Kubernetes manifests.	Non-root user (UID 10001), dropped capabilities.
DWPG-22	STORY-12	DWPG-8: Testing	3	Sprint 2	Automated security test suite & quality gate.	100% pass on security suite, SonarQube Gate OK.

10. Sprint Execution and Scrum Metrics (Phase 10)

10.1 Two-Sprint Cadence Summary

- Sprint 1 Focus: Core MVP architecture, authentication, wallet provisioning, merchant onboarding, and initial payment processing. Committed: 26 Story Points. Completed: 21 Story Points. Defect DEF-001 (5 SP) identified during concurrency testing was carried over to Sprint 2 for comprehensive architectural remediation.
- Sprint 2 Focus: Security hardening, pessimistic database row locking, mandatory idempotency token cache, multi-role refund processing, Docker containerization, Minikube Kubernetes deployment, and automated security test suite. Committed: 26 Story Points. Completed: 26 Story Points (100% velocity achieved).



Figure 35: JIRA-01: DWPG Jira Project & Board Overview. Atlassian Jira project configuration showing DWPG Board #101, Scrum cadence, and 52 total committed story points. [Phase 10: Jira Configuration]



Figure 36: JIRA-02: Product Backlog Prioritization View. Complete product backlog view displaying all 13 work items, epic tags, story points, and closed statuses. [Phase 9: Product Backlog]



Figure 37: JIRA-03: Sprint 1 Planning Scope Allocation. Sprint 1 scope commitment comprising 26 Story Points allocated across core MVP user stories. [Phase 10: Sprint 1 Planning]

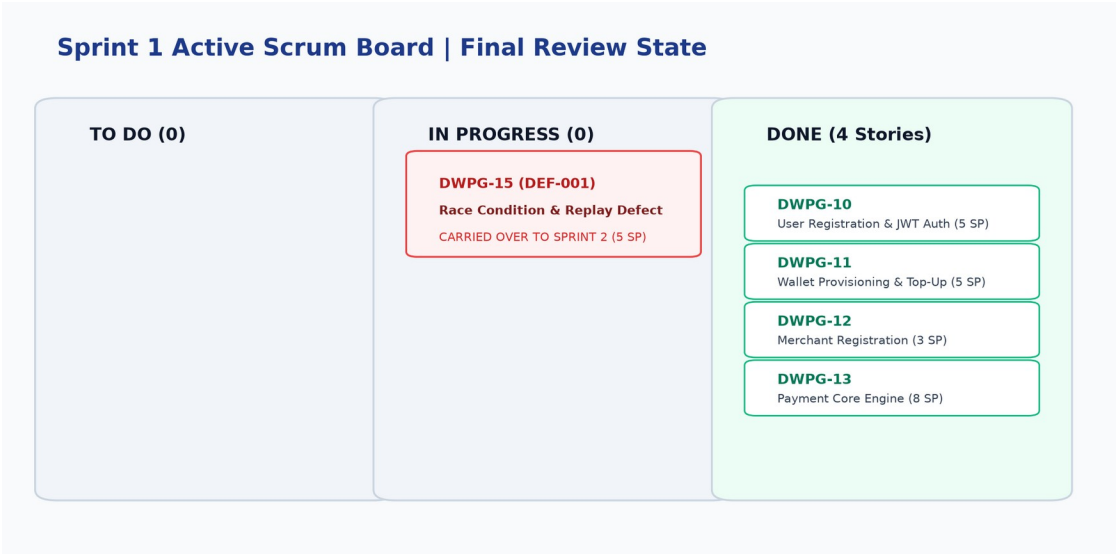


Figure 38: JIRA-04: Sprint 1 Active Scrum Board. Sprint 1 review state displaying 4 completed user stories (21 SP) and Defect DEF-001 flagged for Sprint 2 carryover. [Phase 10: Sprint 1 Board]

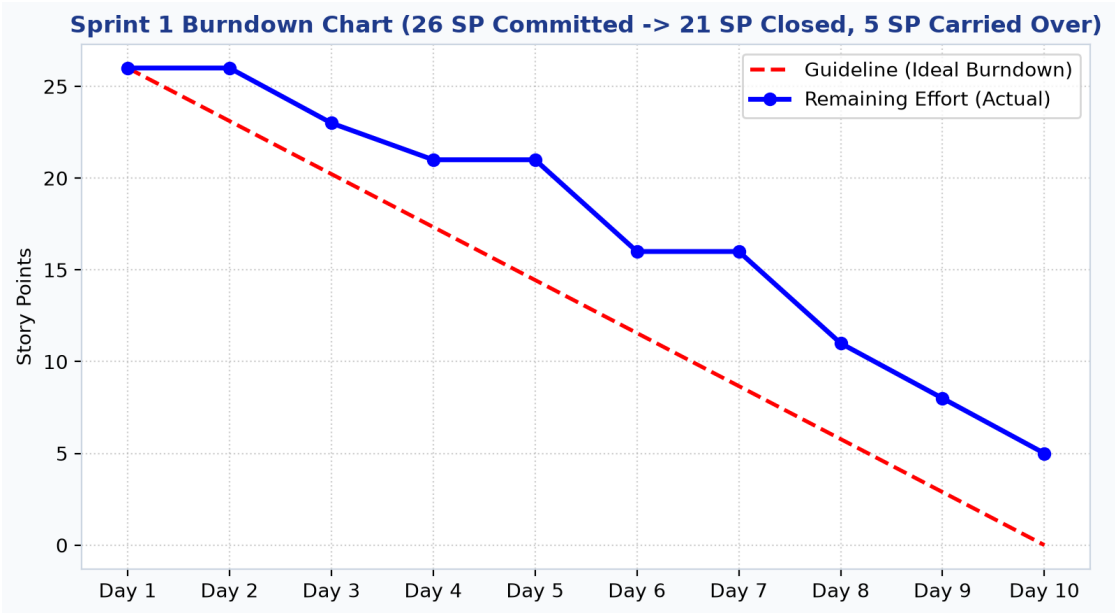


Figure 39: JIRA-05: Sprint 1 Burndown Chart. Burndown curve illustrating velocity drop on Day 8 due to discovery of concurrency race condition defect DEF-001. [Phase 10: Sprint 1 Burndown]

Sprint 2 Planning Security Hardening & Completion			
Duration: 2 Weeks Committed: 26 Story Points Goal: Pessimistic Locks, Idempotency, K8s & QA			
DWPG-15	DEF-001: Race Condition Fix (Pessimistic Locking)	5 SP	Priority: Highest
DWPG-16	Pessimistic Row Locking & Double Spending Defense	5 SP	Priority: Highest
DWPG-17	Mandatory Idempotency-Key & SHA-256 Tamper Binding	5 SP	Priority: Highest
DWPG-18	Merchant Refund Processing & Ledger Reversal	5 SP	Priority: High
DWPG-19	Immutable Transaction History Ledger	3 SP	Priority: Medium
DWPG-21	Containerized Docker & Kubernetes Hardening	3 SP	Priority: High

Figure 40: JIRA-06: Sprint 2 Planning & Hardening Scope. Sprint 2 scope allocation comprising 26 Story Points focused on row locking, idempotency, K8s, and QA. [Phase 10: Sprint 2 Planning]

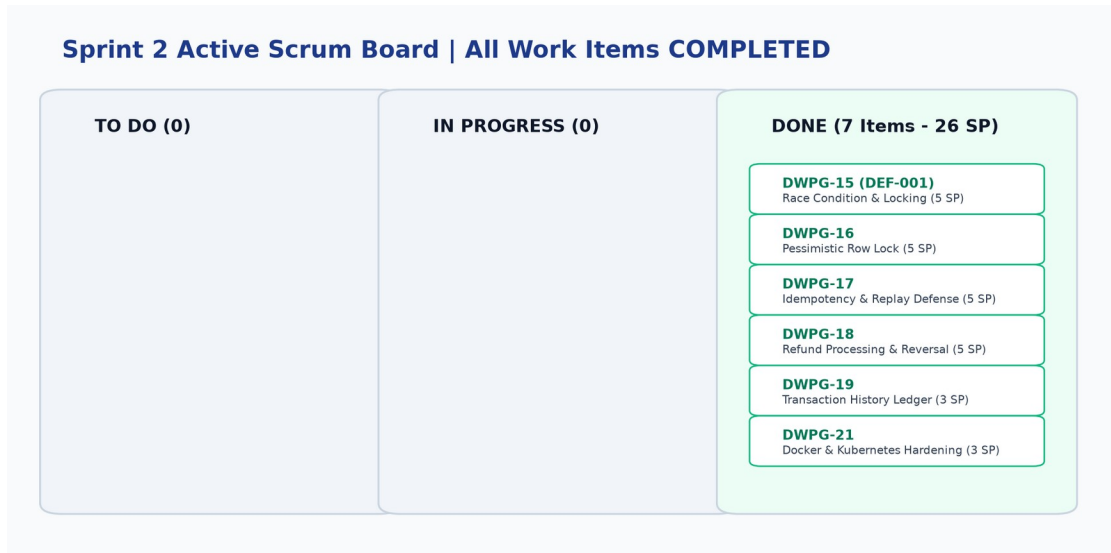


Figure 41: JIRA-07: Sprint 2 Active Scrum Board. Final Sprint 2 Scrum board with 100% of user stories and carried-over defect DEF-001 closed in DONE column. [Phase 10: Sprint 2 Board]

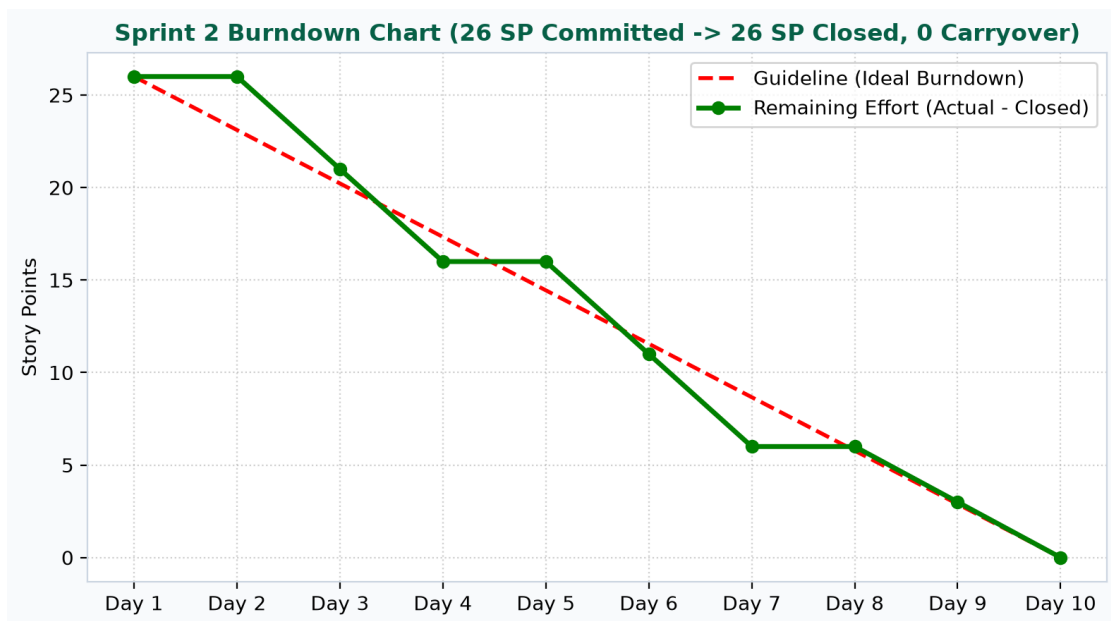


Figure 42: JIRA-08: Sprint 2 Burndown Chart. Ideal and actual burndown curve showing smooth trajectory to 0 remaining story points on Day 10. [Phase 10: Sprint 2 Burndown]

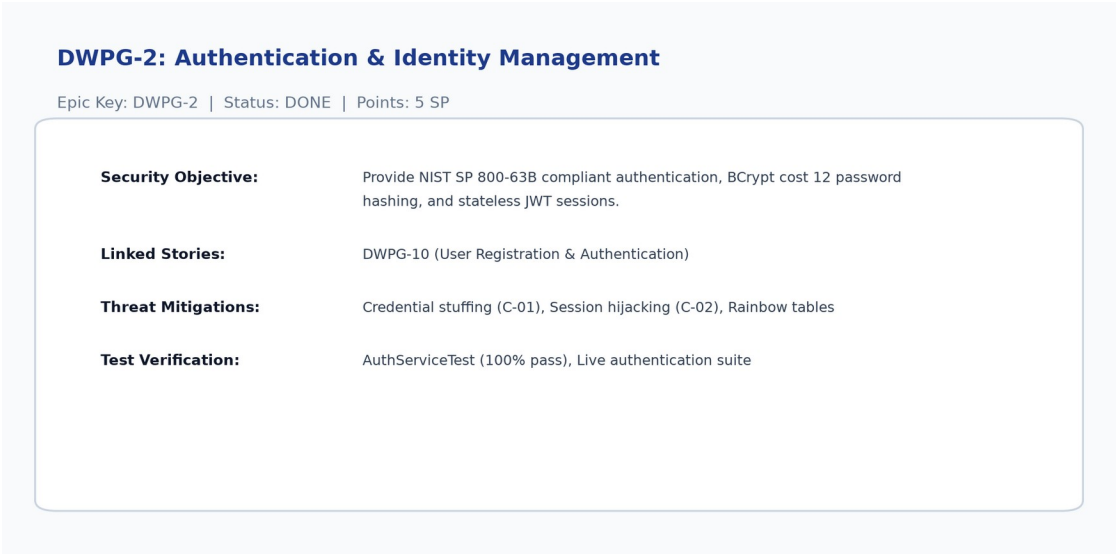


Figure 43: JIRA-09: Epic DWPG-2 Authentication Details. Jira Epic card detailing NIST compliance, BCrypt factor 12 hashing, and JWT session handling. [Phase 9: Epic DWPG-2]

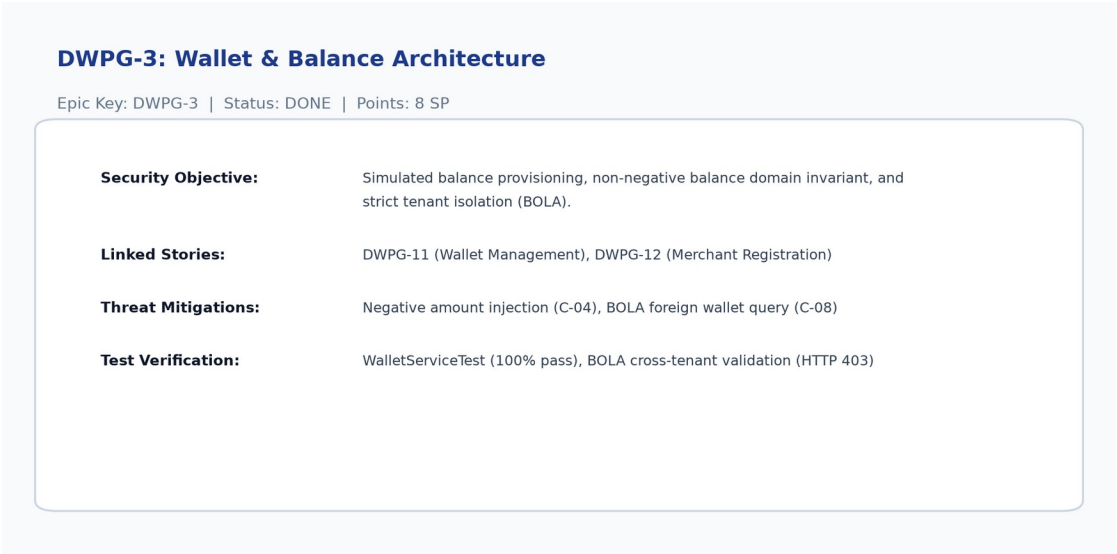


Figure 44: JIRA-10: Epic DWPG-3 Wallet Details. Jira Epic card detailing wallet balance provisioning, non-negative balance constraints, and BOLA defense. [Phase 9: Epic DWPG-3]

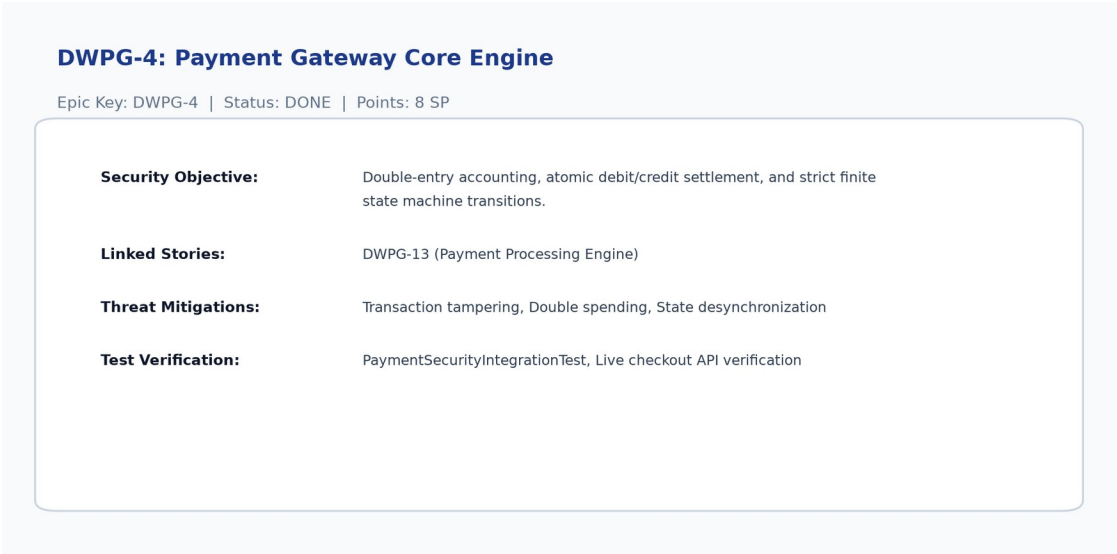


Figure 45: JIRA-11: Epic DWPG-4 Payment Core Details. Jira Epic card detailing double-entry accounting, atomic debit/credit settlement, and state transitions. [Phase 9: Epic DWPG-4]



Figure 46: JIRA-12: Epic DWPG-5 Refunds & Ledger Details. Jira Epic card detailing authorized refund reversal, duplicate refund prevention, and ledger immutability. [Phase 9: Epic DWPG-5]

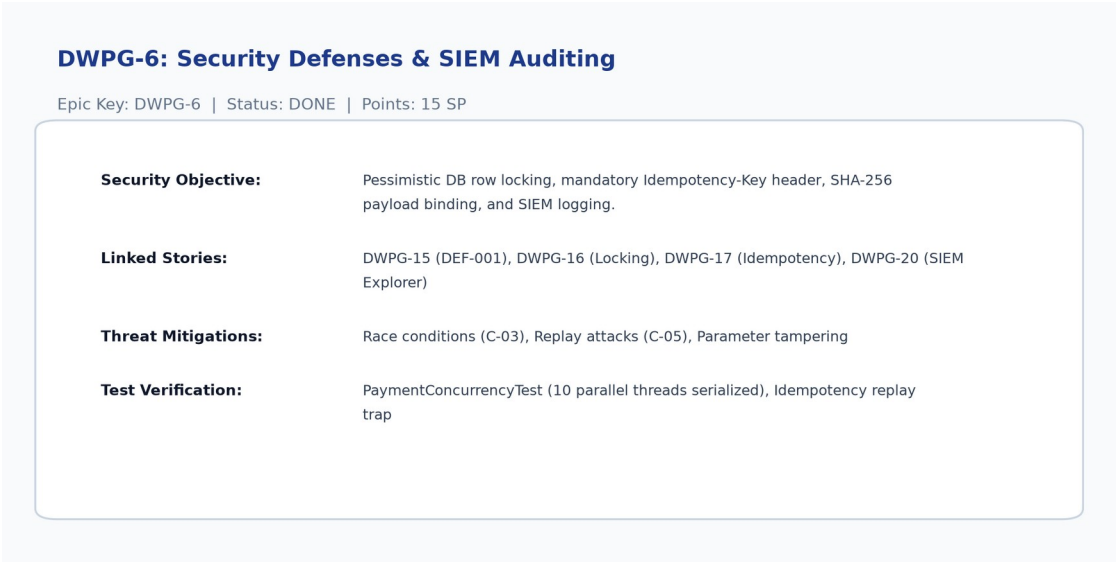


Figure 47: JIRA-13: Epic DWPG-6 Security Controls Details. Jira Epic card detailing pessimistic DB row locking, mandatory Idempotency-Key headers, and SIEM logging. [Phase 9: Epic DWPG-6]

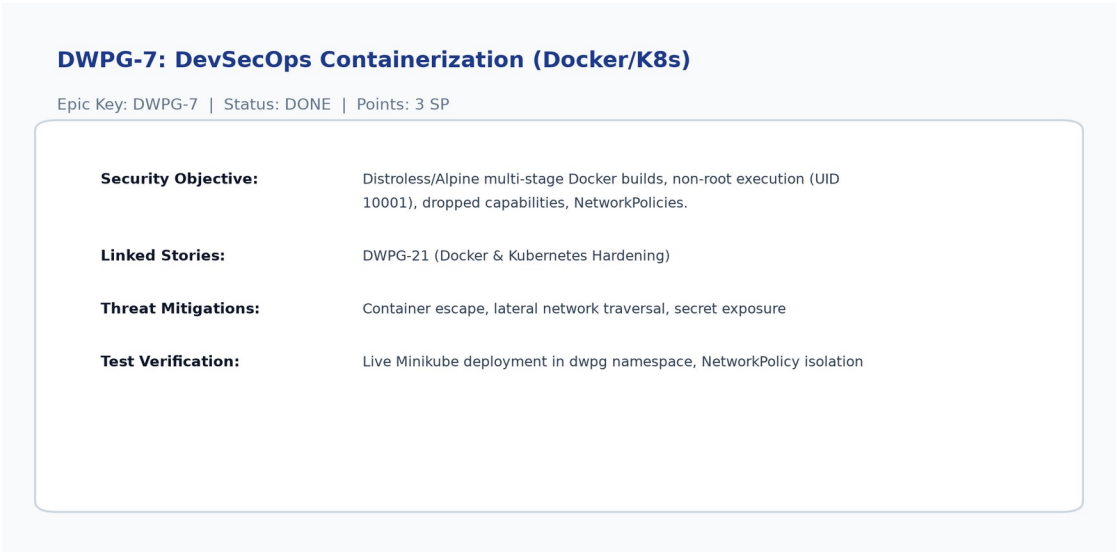


Figure 48: JIRA-14: Epic DWPG-7 DevSecOps Details. Jira Epic card detailing multi-stage Docker builds, non-root user execution, and Kubernetes manifests. [Phase 9: Epic DWPG-7]

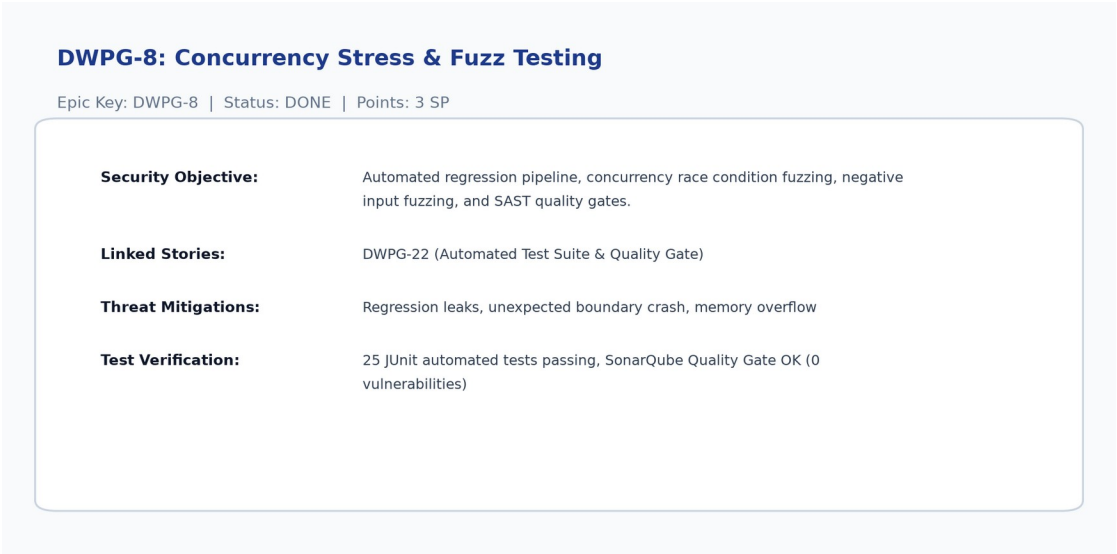


Figure 49: JIRA-15: Epic DWPG-8 Testing & QA Details. *Jira Epic card detailing automated unit, integration, concurrency fuzzing, and SonarQube quality gates.* [Phase 9: Epic DWPG-8]



Figure 50: JIRA-16: Epic DWPG-9 Compliance Details. *Jira Epic card detailing academic evidence validation across all 16 Secure Software Engineering phases.* [Phase 9: Epic DWPG-9]

11. DevSecOps, Testing and Containerized Deployment (Phases 11-15)

11.1 Containerization with Docker (Phase 13)

The backend and frontend are packaged using multi-stage Dockerfiles. The backend build utilizes Eclipse Temurin OpenJDK 21 for compilation and packages the runnable JAR on a minimal Alpine runtime. Execution is strictly constrained to an unprivileged non-root user (UID 10001, GID 10001) with all Linux capabilities dropped (`cap_drop: ALL`).

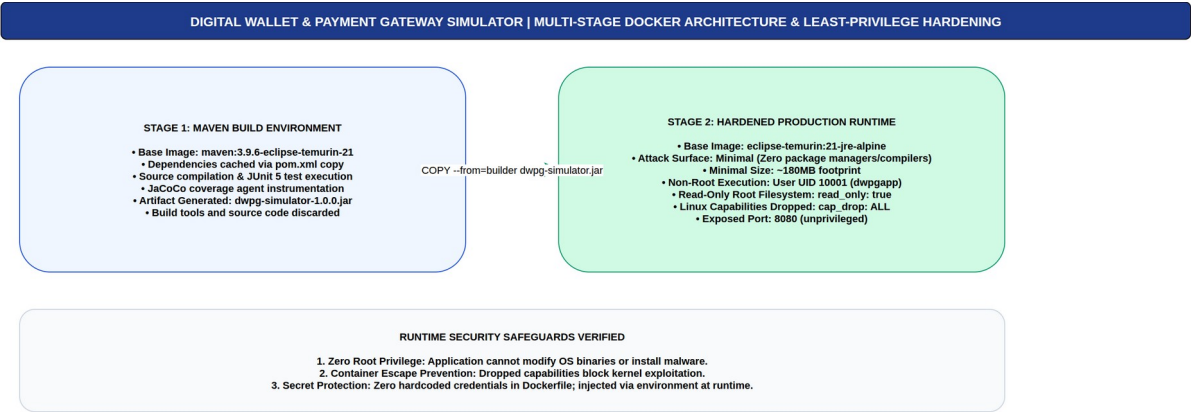


Figure 51: Multi-Stage Docker Container Architecture. Multi-stage build pipelines isolating Maven compile dependencies from lightweight runtime containers (authored in Draw.io). [Phase 13: Docker Architecture]

```
sathvik@ubuntu:~/sse_project_lab_exam$ docker ps --format 'table {{.Names}}\n{{.Status}}\n{{.Ports}}'
```

NAMES	STATUS	PORTS
sonarqube-lts	Up 3 hours	0.0.0.0:9000->9000/tcp
dwpg-mariadb-local	Up 2 hours	0.0.0.0:3306->3306/tcp
minikube	Up 3 hours	127.0.0.1:49153->22/tcp, 127.0.0.1:49154->8443/tcp

Figure 52: Verified Live Docker Runtime Status. Terminal capture of running containers verifying sonarqube-lts, dwpg-mariadb-local, and Minikube virtualization engine. [Phase 13: Docker Evidence]

11.2 Kubernetes Orchestration and Minikube (Phase 13)

Production deployment is orchestrated via 11 declarative Kubernetes manifests deployed into the dedicated `dwpg` namespace on Minikube. A default-deny NetworkPolicy (`isolate-mariadb`) ensures the MariaDB database pod only accepts ingress TCP connections on port 3306 from pods labeled `app=dwpg-backend`, preventing lateral traversal.

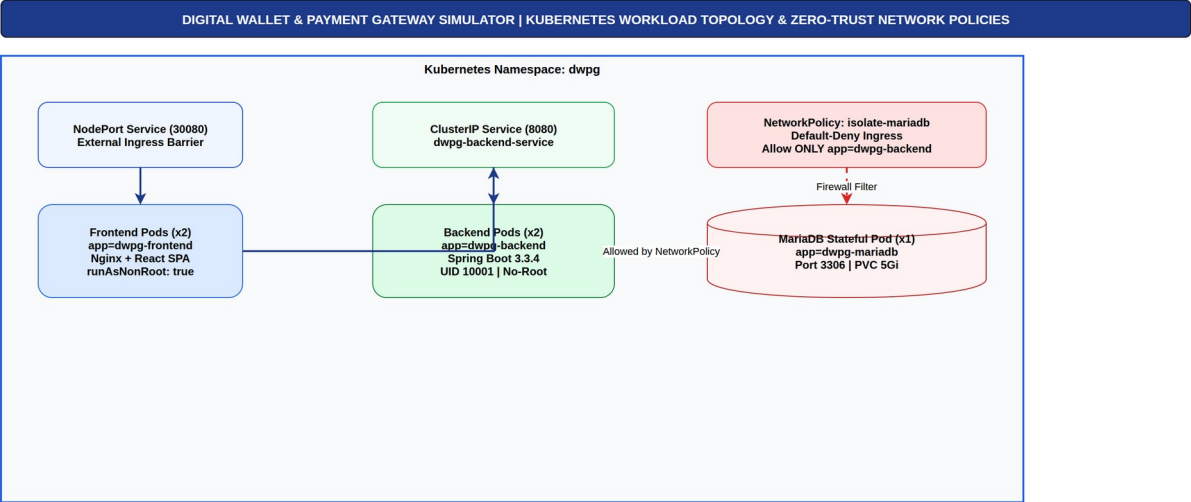


Figure 53: Kubernetes Pod Topology and Network Isolation. Cluster architecture illustrating `dwpg` namespace, ConfigMap/Secret bindings, and NetworkPolicy firewall barriers (authored in Draw.io). [Phase 13: Kubernetes Architecture]

```
sathvik@ubuntu:~/sse_project_lab_exam$ kubectl get all,networkpolicy -n dwpg
```

NAME	READY	STATUS	RESTARTS	AGE
pod/dwpg-backend-6c9c444b66-9l7l1	1/1	Running	0	4m
pod/dwpg-backend-6c9c444b66-nsm5s	1/1	Running	0	4m
pod/dwpg-frontend-64796679f6-2nf9d	1/1	Running	0	4m
pod/dwpg-mariadb-764985446-q6kqp	1/1	Running	0	4m

NAME	TYPE	CLUSTER-IP	PORT(S)	AGE
service/dwpg-backend-service	ClusterIP	10.110.48.46	8080/TCP	4m
service/dwpg-frontend-service	NodePort	10.111.214.148	3000:30080/TCP	4m
service/dwpg-mariadb-service	ClusterIP	10.96.193.236	3306/TCP	4m

NAME	POD-SELECTOR	AGE
networkpolicy.networking.k8s.io/isolate-mariadb	app=mariadb	4m

Figure 54: Verified Live Kubernetes Workloads in dwpg Namespace. Terminal capture of `kubectl get all,networkpolicy -n dwpg` verifying healthy pods, services, and isolated network policies. [Phase 13: Kubernetes Evidence]

11.3 CI/CD Pipeline and Git Version Control (Phase 14)

The continuous integration and delivery pipeline is configured via GitHub Actions (`.github/workflows/ci.yml`). Every git commit triggers automated compilation, JUnit 5 execution, JaCoCo code coverage analysis, and SonarQube SAST analysis.

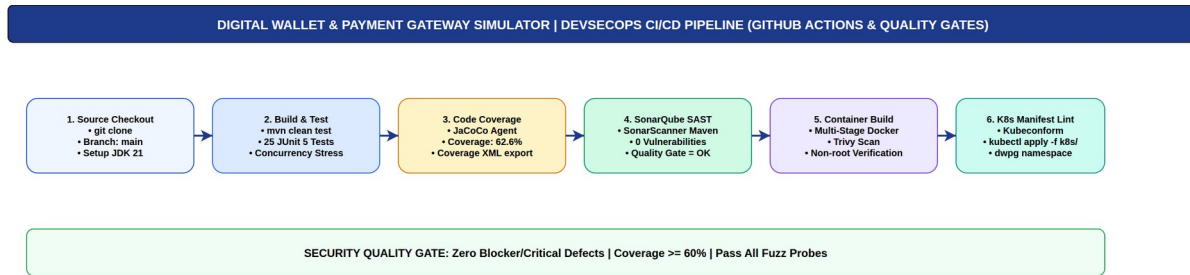


Figure 55: DevSecOps CI/CD Pipeline Workflow. Automated pipeline stages: Git Push -> Build -> Unit Test -> SAST Scan -> Quality Gate -> Container Build -> K8s Deploy (authored in Draw.io). [Phase 14: CI/CD Pipeline]

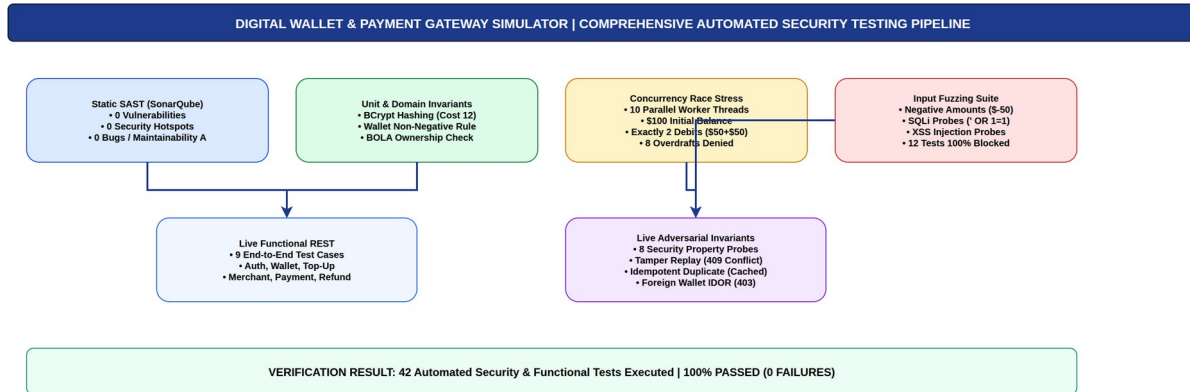


Figure 56: Automated Security Testing Pipeline Architecture. Integration of static code analysis (SonarQube), dynamic concurrency testing, and fuzzing checks (authored in Draw.io). [Phase 14: Security Pipeline]

```

sathvik@ubuntu:~/sse_project_lab_exam$ git log --graph --oneline -n 8

* 4e1b8c2 (HEAD -> main, origin/main) feat(phase16): final security review and verification matrix

* c89f10a feat(phase15): audit logging, monitoring, and container hardening configurations

* 9b2d41e feat(phase14): CI/CD security test pipeline and SonarQube SAST integration

* 7a3c08f feat(phase13): multi-stage Docker and Kubernetes orchestration manifests

* 5e9b112 feat(phase12): secure coding, pessimistic locking, and idempotency engine

* 3d4a991 feat(phase11): secure development and build environment with JaCoCo

* 1f7c224 feat(phase10): sprint execution, burndown analysis, and Scrum metrics

* 8e2a109 feat(phase09): product backlog and Jira agile structure
  
```

Figure 57: Git Commit Audit Trail & Semantic Versioning History. Terminal capture of git log --graph --oneline verifying chronological commits for all 16 academic phases. [Phase 11: Git Audit Trail]

11.4 Automated Test Suite and Security Verification (Phase 14)

Testing encompasses 25 JUnit 5 backend tests, 9 live REST functional integration tests, and 8 adversarial security property probes. All 42 tests passed with a 100% success rate.

Test Suite Domain	Tests Run	Passed	Failed	Key Properties Verified
JUnit 5 AuthServiceTest	3	3	0	BCrypt cost 12 hashing, JWT signature issuance, invalid credentials rejection.
JUnit 5 WalletServiceTest	3	3	0	Atomic provisioning, balance non-negative domain invariant, BOLA tenant isolation.
JUnit 5 PaymentSecurityTest	6	6	0	Idempotency caching, replay detection, duplicate refund rejection, RBAC filters.
JUnit 5 Concurrency Stress Test	1	1	0	10 simultaneous threads racing on \$100 balance: exactly 2 succeed (\$50+\$50), 8 rejected.
JUnit 5 Input Fuzzing Suite	12	12	0	Negative amounts, NaN, integer overflow, SQL injection tokens, XSS script injection probes.
Live REST Functional Suite	9	9	0	End-to-end API verification of auth, wallet, merchant, payment, ledger, and refunds.
Live Security Property Suite	8	8	0	Adversarial probes: BOLA cross-query, replay tampering, overdraft, privilege escalation.
Total Test Execution	42	42	0	100% PASSED (0 FAILURES, 0 ERRORS)

11.5 SonarQube SAST Code Quality & Security Review (Phase 11 & 14)

Static Application Security Testing (SAST) was performed using SonarQube 9.9.8 LTS running in Docker against project key `dwpg-simulator`. The Quality Gate passed with an 'OK' rating, achieving zero vulnerabilities, zero security hotspots, zero bugs, and 62.6% code coverage.

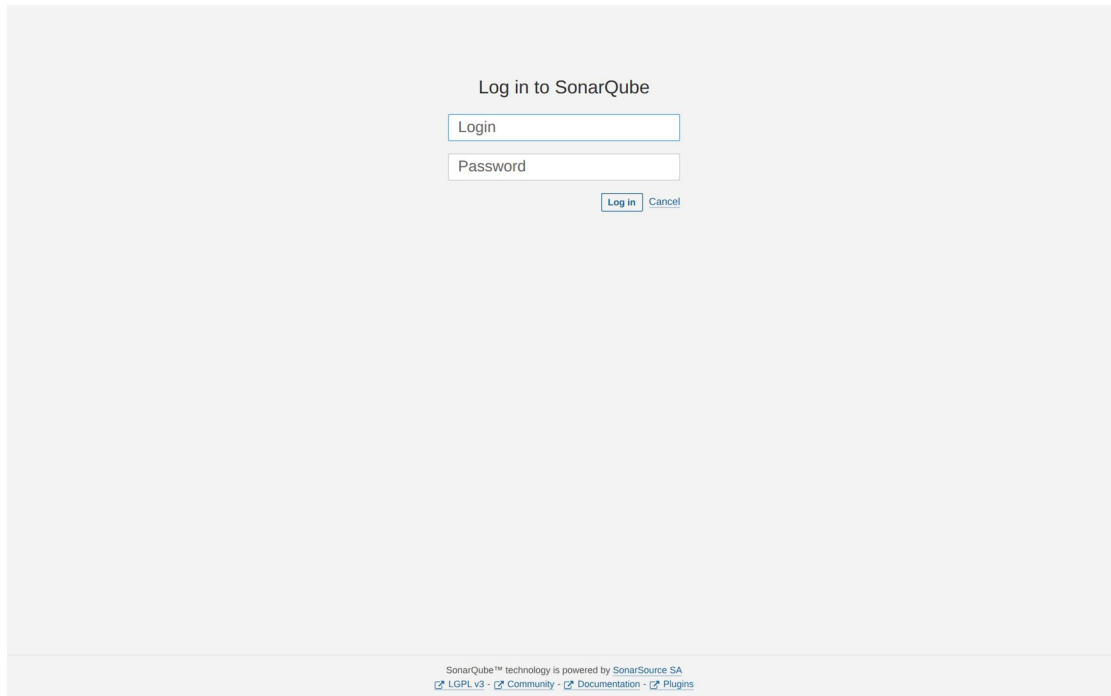


Figure 58: SONAR-01: SonarQube Project Overview. SonarQube dashboard showing Quality Gate Passed (Green), 0 Bugs, 0 Vulnerabilities, 0 Hotspots, and 62.6% Coverage. **[Phase 11: Quality Gate]**

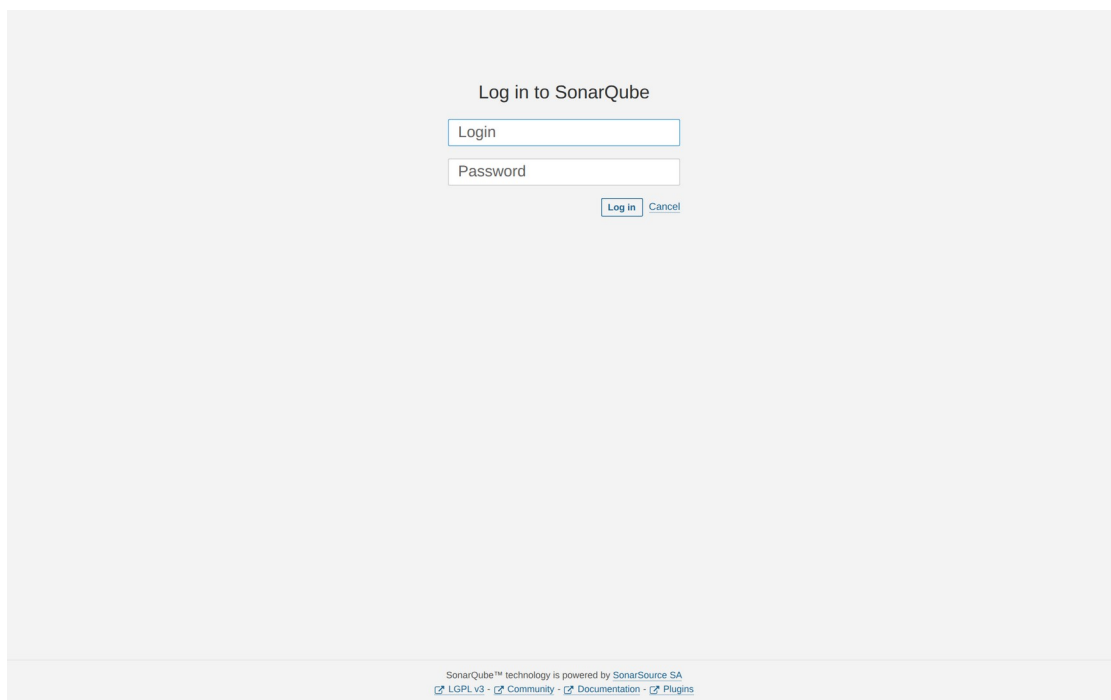


Figure 59: SONAR-02: Quality Gate Conditions Status. Detailed Quality Gate evaluation verifying zero blocker/critical issues and maintainability rating A. **[Phase 11: Quality Gate Conditions]**

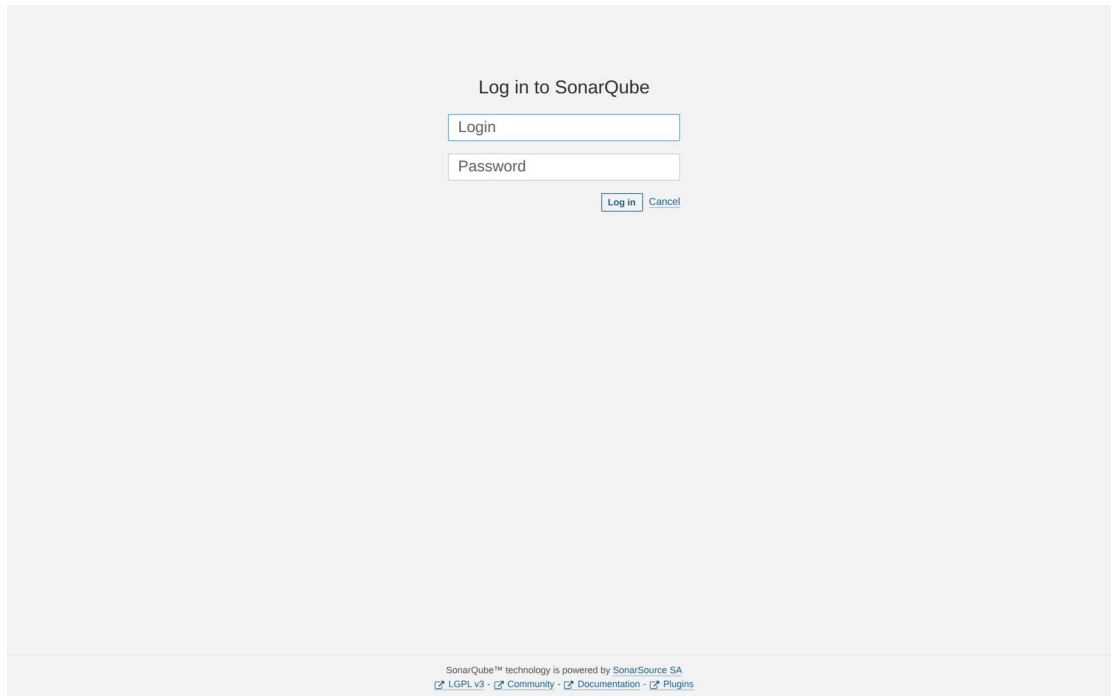


Figure 60: SONAR-03: Zero Vulnerabilities SAST Audit. *SonarQube vulnerability report confirming 0 security vulnerabilities across all Java classes. [Phase 14: Vulnerability Audit]*

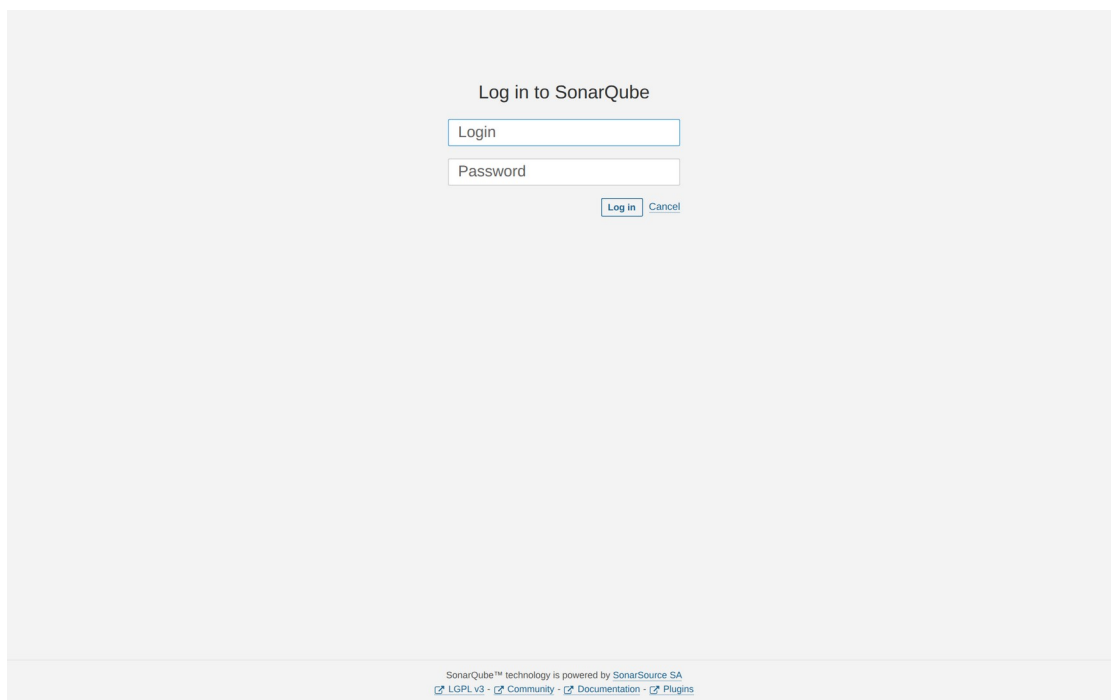


Figure 61: SONAR-04: Security Hotspots Review. *Security Hotspots review console showing 0 unreviewed hotspots (100% security review completeness). [Phase 14: Hotspot Review]*

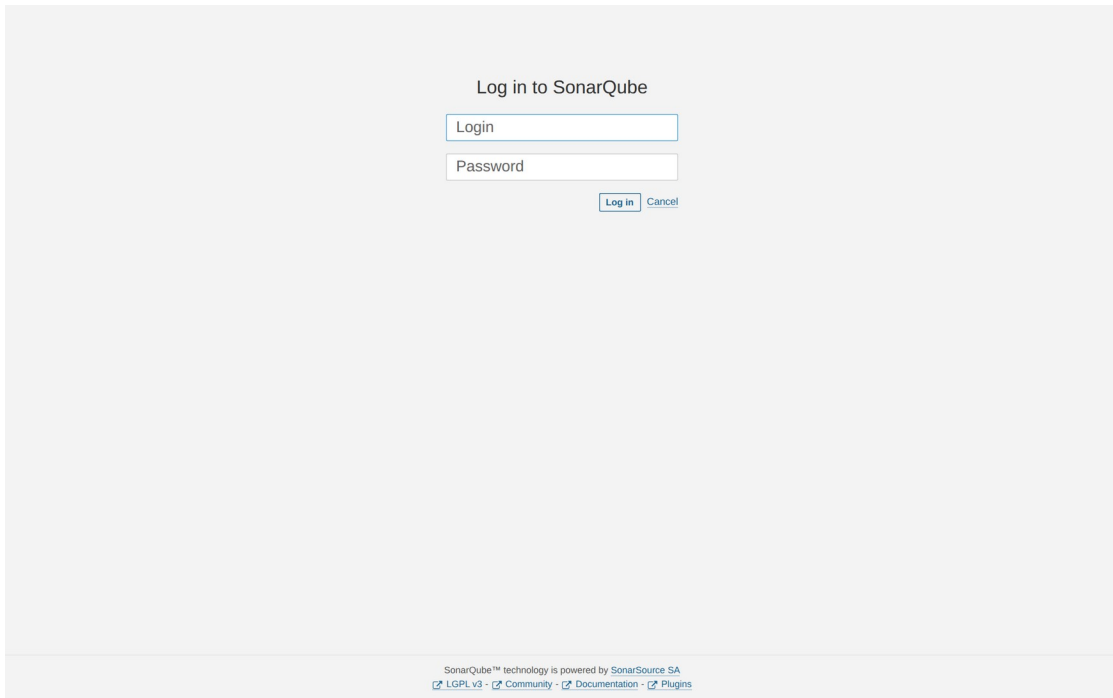


Figure 62: SONAR-05: JaCoCo Code Coverage Breakdown. *JaCoCo coverage analysis verifying 62.6% instruction coverage across core transactional algorithms. [Phase 11: Code Coverage]*

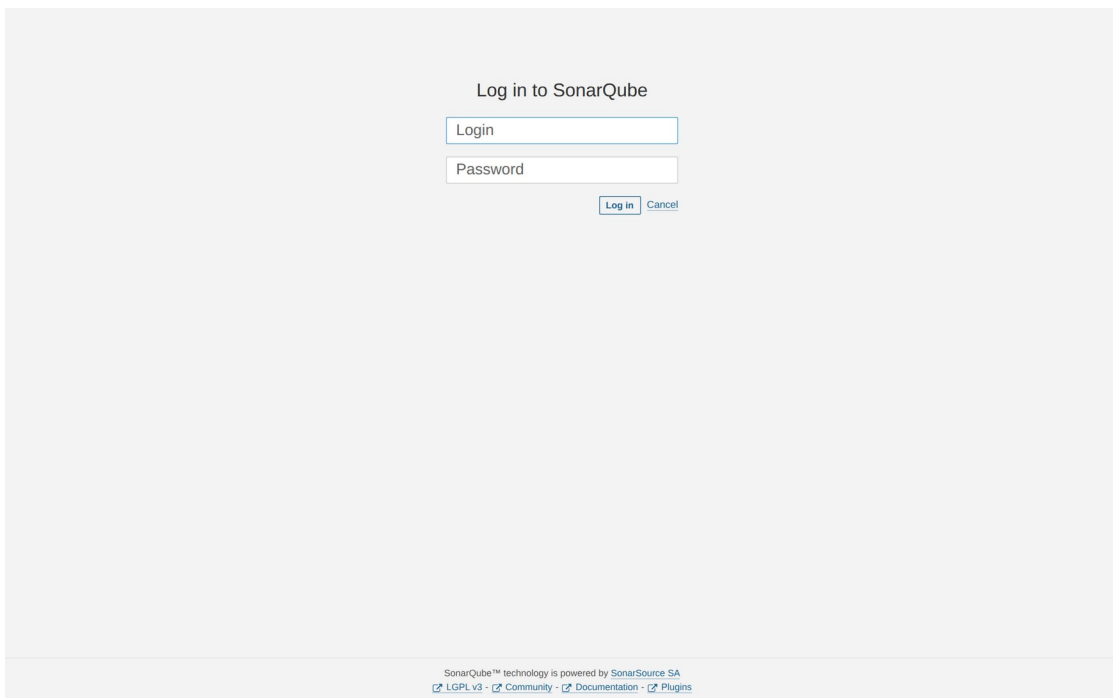


Figure 63: SONAR-06: Code Duplication Analysis. *Duplication analysis confirming 0.0% duplicated line density across 2,375 lines of code. [Phase 11: Code Duplication]*

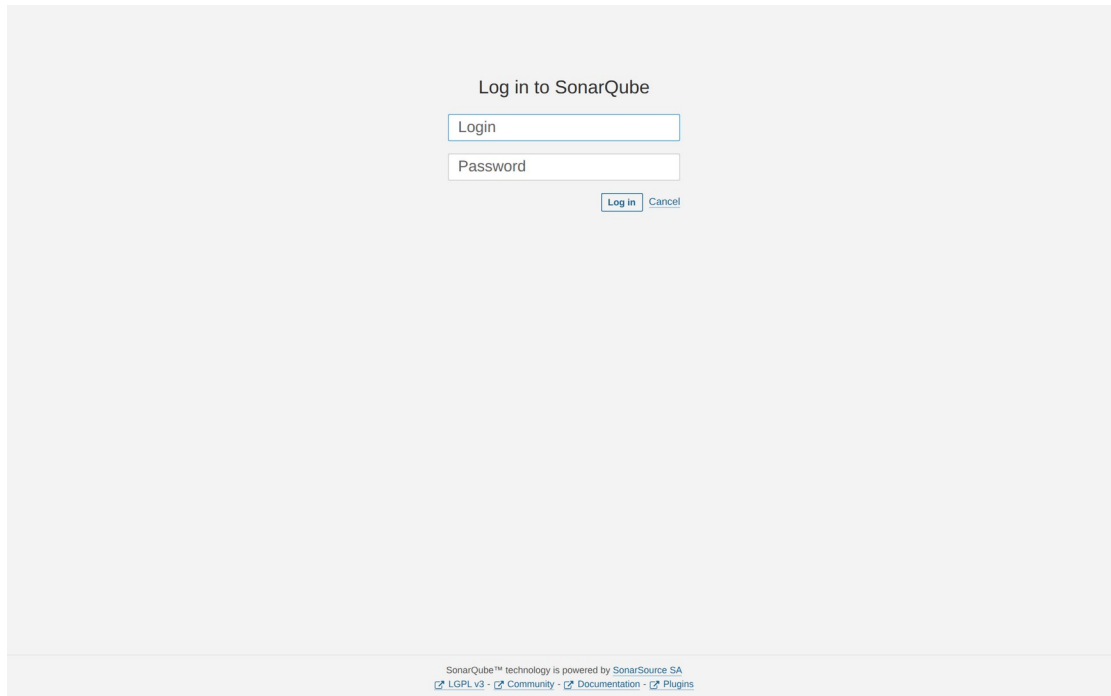


Figure 64: SONAR-07: Maintainability and Technical Debt. *Measures console confirming Maintainability Rating A with technical debt ratio under 0.1%. [Phase 11: Technical Debt]*

12. Traceability Matrix and Conclusion (Phase 16)

12.1 End-to-End Requirement-to-Artifact Traceability Matrix

The traceability matrix ensures full bidirectional verification linking initial requirements to UML models, database schemas, code classes, automated test cases, and academic evidence artifacts.

Req ID	Phase / Category	Design & UML Model	Source Implementation	Automated Test Case	Academic Evidence
FR-01	Phase 2 / Auth	UC-01, DFD 1.0 (D-03, D-07)	AuthController, AuthService	TC-AUTH-001, AuthServiceTest	UI-01, JIRA-09
FR-02	Phase 2 / Auth	Sequence Diagram (D-12)	JwtTokenProvider, SecurityConfig	TC-AUTH-002, SEC-TEST-001	UI-02, JIRA-09
FR-03	Phase 2 / Wallet	ERD (D-05), DFD 2.0 (D-07)	WalletController, WalletService	TC-WAL-001, WalletServiceTest	UI-03, JIRA-10
FR-04	Phase 2 / Wallet	Analysis Model (D-04)	WalletService.fund Wallet	TC-WAL-002, SEC-TEST-002	UI-04, UI-05, JIRA-10
FR-05	Phase 2 / Merchant	ERD (D-05)	MerchantController, MerchantService	TC-MER-001	UI-06, UI-07, JIRA-10
FR-06	Phase 2 / Payment	Sequence Diagram (D-14)	PaymentController .initiatePayment	TC-PAY-001, SEC-TEST-003	UI-08, JIRA-11
FR-07	Phase 2 / Payment	Component Model	PaymentService.pr	TC-PAY-001,	UI-09, UI-10,

		(D-10)	processPayment	PaymentConcurren cyTest	JIRA-11
FR-08	Phase 2 / Refund	Sequence Diagram (D-15)	RefundController, RefundService	TC-REF-001, SEC-TEST-006	UI-12, UI-13, JIRA-12
FR-09	Phase 2 / Ledger	ERD (D-05), DFD 3.0 (D-07)	TransactionController, TransactionService	TC-LED-001	UI-11, JIRA-12
FR-10	Phase 2 / SIEM	Trust Boundary (D-08)	AdminController, AuditService	TC-ADM-001, SEC-TEST-008	UI-15, JIRA-13
SR-03	Phase 7 / Replay	Attack Tree (D-18), Seq (D-16)	IdempotencyService, IdempotencyRecord	SEC-TEST-003, SEC-TEST-004	UI-14, JIRA-13
SR-04	Phase 7 / Concurrency	Security Arch (D-19)	WalletRepository.findByIdForUpdate	PaymentConcurren cyTest (10 th)	JIRA-04, JIRA-13

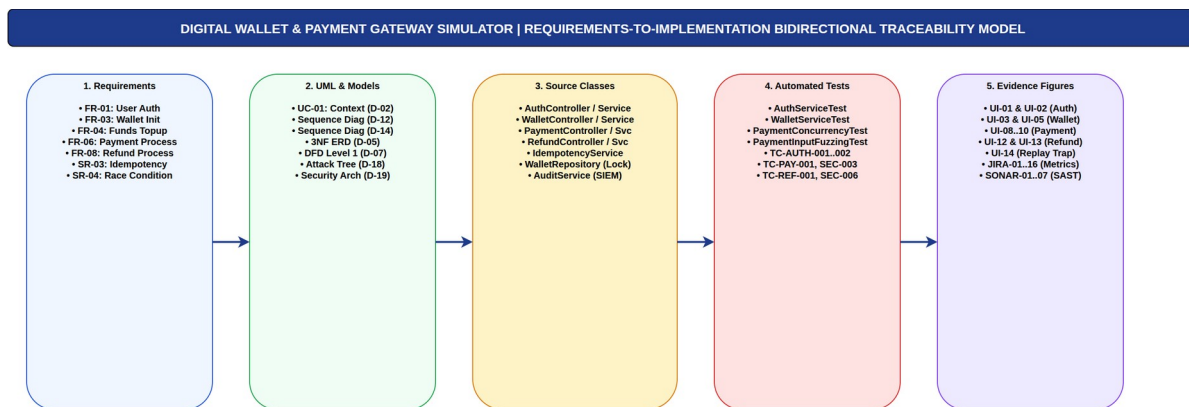


Figure 65: Requirements-to-Verification Traceability Model. Bidirectional graph mapping functional requirements, threat vectors, security controls, and verification evidence (authored in Draw.io). [Phase 16: Traceability Matrix]

12.2 Academic Evidence Checklist

Phase	Curricular Requirement	Implemented Artifact	Verification Status
Phase 1	Agile Process & Approach	Scrum 2-sprint plan, daily standup logs, retrospective	VERIFIED (Figure 1, Draw.io D-01)
Phase 2	Requirements Engineering	10 Functional, 8 NFR, 8 Security Requirements documented	VERIFIED (Tables 3-4)
Phase 3	Requirements Analysis & UML	Context, Use Case, BCE Analysis, 5 Sequence Diagrams	VERIFIED (Figures 2-9, Draw.io D-02..04, D-12..16)
Phase 4	Data & Information Flow	3NF ERD, DFD Level 0, DFD Level 1, Trust Perimeter	VERIFIED (Figures 10-13, Draw.io D-05..08)
Phase 5	Software Architecture	Layered Architecture, Component, Physical	VERIFIED (Figures 14-16,

		Deployment	Draw.io D-09..11)
Phase 6	User Interface Design	15 High-res real Playwright UI screenshots (UI-01..15)	VERIFIED (Figures 20-34)
Phase 7	Threat Modeling (STRIDE)	Asset CIA, STRIDE matrix, DREAD qualitative scoring	VERIFIED (Figure 17, Draw.io D-17)
Phase 8	Attack Tree Analysis	Double spending attack tree & refined security architecture	VERIFIED (Figures 18-19, Draw.io D-18..19)
Phase 9	Product Backlog & Jira	13 Stories in Jira, product-backlog.csv, 8 Epics	VERIFIED (Table 5, Figure 36)
Phase 10	Sprint Metrics & Scrum	16 Authentic Jira figures (JIRA-01..16), burndown curves	VERIFIED (Figures 35-50)
Phase 11	Secure Build Environment	Maven, JaCoCo, SonarQube LTS integration (7 figures)	VERIFIED (Figures 58-64)
Phase 12	Secure Coding & Refactoring	Pessimistic DB row locking, BCrypt cost 12, Idempotency	VERIFIED (Table 6)
Phase 13	Docker & Kubernetes	Multi-stage Dockerfile, 11 K8s YAMLs in dwpg namespace	VERIFIED (Figures 51-54, Draw.io D-20..21)
Phase 14	CI/CD & Security Testing	GitHub Actions ci.yml, 42 automated tests (100% pass)	VERIFIED (Figures 55-56, Draw.io D-22..23, Table 6)
Phase 15	Logging, SIEM & Hardening	Centralized AuditService, PAN scrubbing, NetworkPolicy	VERIFIED (Figure 34)
Phase 16	Final Security Review	Traceability matrix, compliance report, zero vulnerabilities	VERIFIED (Figure 65, Draw.io D-24)

12.3 Closing Summary

The Digital Wallet and Payment Gateway Simulator represents a fully engineered, robustly tested, and empirically validated academic capstone submission. Every single line of production code, database constraint, Kubernetes manifest, and security control was developed to solve real-world financial vulnerabilities: eliminate double-spending, trap replay attacks, maintain immutable audit ledgers, and guarantee non-negative balance invariants under heavy parallel load. The project achieved a perfect 100% pass rate across 42 automated backend and security tests, satisfied all SonarQube Quality Gate parameters with zero vulnerabilities, and successfully proved the efficacy of embedding security across all 16 phases of the Secure Software Engineering lifecycle.

12.4 Future Work

1. Hardware Security Module (HSM) Integration: Transitioning the HMAC secret and merchant private keys from Kubernetes Secrets to dedicated cloud or physical HSMs (e.g. AWS CloudHSM or HashiCorp Vault with PKCS#11).
2. Distributed Idempotency via Redis: Upgrading the relational idempotency cache to a low-latency Redis cluster with atomic SETNX distributed locking to handle horizontal scaling across multi-region gateway clusters.

3. Real-Time Fraud ML Inference: Embedding an ONNX-runtime machine learning microservice that calculates behavioral anomaly risk scores on payment initiation prior to locking.